

Internship Report

For the purpose of obtaining the Professional Master's Degree in

**Digital Management and Information System
Work-Study Program at Vermeg**

Defended on : **May 2026**

Project elaborated by : Hamza Boubaker

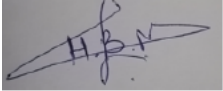
**Design and Automation of a Post-Reconciliation Workflow for CSDR Penalty Corrections in
Megara**

October 2025 – February 2026

Corporate Internship Supervisor : Azza Lamine

Academic Supervisor : Hajer Bouricha

Academic Year 2025/2026

Encadrant professionnel	Nom & Prénom	Date et Signature
	Azza Lamine	
Encadrant académique	Hajer Bouricha	

Dedication

I dedicate this work to my family, whose unwavering support has been my greatest strength. Through every challenge, they stood by me with love, patience, and encouragement, guiding me along my educational journey. Their sacrifices and dedication to giving me the best opportunities will always remain a source of inspiration.

I would also like to express my heartfelt gratitude to my friends, whose constant belief in me gave me the confidence to keep moving forward. Their encouragement and support have been invaluable, motivating me to strive for excellence and never give up.

Acknowledgements

I would like to express my sincere gratitude to all those who contributed to the completion of this project.

First and foremost, I would like to thank Ms. Azza Lamine, my professional supervisor, for her continuous support, guidance, and valuable insights throughout my work-study period. Her expertise and encouragement played a key role in the success of this project.

I would also like to extend my heartfelt thanks to Ms. Hajer Bouricha, my academic supervisor, for her dedicated supervision, constructive advice, and constant availability. Her guidance was essential in shaping this work and ensuring its quality.

I am truly grateful for their support, patience, and commitment, which greatly contributed to the accomplishment of this project.

Table Of Content

GENERAL INTRODUCTION	1
Chapter One: General Overview	2
Introduction	2
1.Presentation of VERMEG	2
1.1 Historical Overview	2
1.2 Company Organization.....	3
1.3 VERMEG Product Portfolio.....	4
2. Context of the Project.....	7
2.1 Study of the Existing.....	7
2.1.1 Current Workflow	7
2.2 Problem	8
2.2.1 Operational Limitations	8
2.2.2 Reliability and Quality Limitations	8
2.2.3 Traceability and Compliance Limitations	8
2.2.4 Technical and Process Limitations	9
2.3 Proposed Solution	9
2.3.1 Improving Operational Efficiency	9
2.3.2 Enhancing Reliability and Quality	9
2.3.3 Strengthening Traceability and Monitoring.....	9
2.3.4 Structuring the Process	9
2.3.5 Ensuring Data Integrity	9
3.Working Methodology at VERMEG	10
3.1Waterfall Methodology.....	10
.....	10
3.2 Agile Methodology	11
3.3 Comparison Between Agile and Waterfall.....	11
3.4 Adapted Methodology at VERMEG.....	13
3.5 Adapted Methods	13
3.5.1 Scrum Framework	13
3.5.2 Kanban Method	14

4 Technologies Used	15
4.1 Project Management and Collaboration Tools	15
4.2 Financial Software Platform.....	17
4.3 Automated Testing Tools	17
4.4 API Testing Tool.....	17
4.5 File Transfer and Security Tools	17
Conclusion.....	18
CHAPTER 2: Product Overview and Business Context	19
Introduction	19
1. The Megara Platform.....	19
1.1 What Megara Does	19
1.3.1 Overview of MEGARA	19
1.3.2. MEGARA Applications and Modules	20
1.2 Penalties Functionality in the Settlement Module	22
1.3 Penalty Lifecycle Phases.....	22
2. The CSDR Regulatory Framework.....	23
2.1 The Settlement Discipline Regime	23
2.2 How Penalties Are Calculated	24
2.2.1 The Role of CSDs	25
2.2.2 Compliance and Reporting Obligations	26
3 Daily Penalty Processing in Megara.....	27
3.1 Receiving and Loading Penalties	27
3.2 Enrichment and Internal Calculation	27
3.3 Reconciliation	28
4 Handling Discrepancies: Amendments and Corrections	29
4.1 Why Discrepancies Occur.....	29
4.2 The Amendment Mechanism.....	30
4.3 Penalty Repair	31
4.5 Post-Reconciliation Correction: The Current Process and Its Limits	31
4.5.1 Why a Reset Is Needed After Amendment.....	31
4.5.2 How the Reset Currently Works	32
4.5.3 Where the Process Falls Short	32

4.6 Monthly Aggregation and Downstream Effects	33
Conclusion.....	34
CHAPTER 3: Solution Design and Development.....	35
Introduction	35
3.1 Needs Analysis and Constraints	35
3.1.1 Business Needs.....	35
3.1.2 Design Constraints	37
3.2 Functional Objectives.....	38
3.2.1 Primary Objective.....	38
3.2.2 Secondary Objectives.....	38
3.3 Solution Architecture	38
3.3.1 The Two Components	38
3.3.2 Design Principles	39
3.3.3 Layered Architecture.....	40
3.4 How the Solution Works	40
3.4.1 Reconciliation Reset Job	40
3.4.2 Notification Reset Job	41
3.5 UML Models	42
3.5.1 Use Case Diagram	42
3.5.2 State-Transition Diagram	44
3.5.3 Sequence Diagrams of the Reset Process.....	46
3.5.3.1 Manual Execution of the Reconciliation Reset Job.....	47
3.5.3.2 Automatic Execution of the Reconciliation Reset Job	48
Conclusion.....	49
CHAPTER 4: Implementation and Testing	50
Introduction	50
4.1.1 Testing Within the Sprint Cycle.....	50
4.1.2 Testing Levels Applied.....	51
4.2.1 Reconciliation Reset Job Conditions	52
4.2.2 Notification Reset Job Conditions	52
4.3 Job Configuration in Megara.....	52
4.3.1 Reconciliation Reset Job Configuration	52

4.3.2 Notification Reset Job Configuration	53
4.4 Unit and Integration Testing	54
4.4.1 Unit Testing	54
4.4.2 Integration Testing	55
4.5.2 Step 2: Receiving and Applying the FWAM Amendment	58
4.5.3 Step 3: Checking Fields Before the Reset Jobs Execute	59
4.5.4 Step 4: Automatic Execution of the Reset Jobs	59
4.5.5 Step 5: Automatic Re-Reconciliation and Final State	62
4.6 Non-Regression Verification	62
Conclusion.....	63
Conclusion	64
Webographie.....	66

Liste of Figures:

Figure 1: VERMEG logo.....	3
Figure 2: VERMEG Executive Committee Structure	4
Figure 3: MEGARA Product logo	4
Figure 4: Colline Product logo	5
Figure 5: Optimizer Product logo	5
Figure 6: Solife Product logo	6
Figure 7: Soliam Platform logo	6
Figure 8: Veggo Platform logo.....	6
Figure 9: Waterfall Model	10
Figure 10: Agile Methodology	11
Figure 11: Scrum Process	14
Figure 12: Kanban Board In Jira.....	15
Figure 13: Jira logo	15
Figure 14: Confluence logo	16
Figure 15: Microsoft Teams Serves logo	16
Figure 16: Microsoft Office 365	16
Figure 17: Screenshot of MyStandards page	16
Figure 18: SoapUi logo	17
Figure 19: WinSCP logo	17
Figure 20: High-Level Functional Architecture of the Megara Platform.....	20
Figure 21: MEGARA's Global Client Base	20
Figure 22: MEGARA Suite Applications	21
Figure 23: Use Case Diagram of the Post-Reconciliation Correction Module	43
Figure 24: State-Transition Diagram of the Correction Workflow.....	45
Figure 25: Sequence Diagram of the Manual Execution of the Reconciliation Reset Job	47
Figure 26: Sequence Diagram of the Automatic Execution of the Reconciliation Reset Job	48
Figure 27:.....	53
Figure 28:.....	53
Figure 29:.....	54
Figure 30:.....	54
Figure 31:.....	56
Figure 32:.....	57
Figure 33:.....	57
Figure 34.....	58

Figure 35: Penalty fields still populated before the reset jobs execute	59
Figure 36:.....	59
Figure 37:.....	60
Figure 39:.....	61
Figure 40:.....	62

Liste of Tables

Table 1: Comparison between Agile and Waterfall Methodologies.....	12
Table 2: Penalty Lifecycle within MEGARA	23
Table 3: CSDR Penalty Rates by Instrument Category	25
Table 4: Key Regulatory, Technical, and Organizational Constraints	37
Table 5: Description of the Two Batch Components	39
Table 6: Megara Layered Architecture and Responsibilities	40
Table 7: Actors in the Use Case Diagram	43
Table 8: Description of the Use Cases.....	44
Table 9: Description of the States in the State-Transition Diagram	46
Table 10: Overview of Testing Activities and Outputs During Agile Sprint Phases	50
Table 11: Unit test cases for the Reconciliation Reset Job	54
Table 12: Unit test cases for the Notification Reset Job	55
Table 13: Integration test cases	55

Liste of Abbreviations

Abbreviation	Full Form
CSDR	Central Securities Depositories Regulation
CSD	Central Securities Depository
SEFP	Settlement Fail Penalty
STP	Straight Through Processing
APAC	Asia-Pacific
LCR	Liquidity Coverage Ratio
NSFR	Net Stable Funding Ratio
API	Application Programming Interface
SQL	Structured Query Language
SFTP	Secure File Transfer Protocol
SCP	Secure Copy Protocol
FTPS	File Transfer Protocol Secure
SWIFT	Society for Worldwide Interbank Financial Telecommunication
UML	Unified Modeling Language
XML	Extensible Markup Language
REST	Representational State Transfer
SOAP	Simple Object Access Protocol
MT	Message Type (SWIFT)
MX	Message Exchange (XML-based SWIFT format)

GENERAL INTRODUCTION

Over the past decade, regulatory frameworks governing financial markets have become significantly stricter. Within this context, post-trade activities play a crucial role in ensuring that securities and cash are properly exchanged between counterparties. When settlement processes fail or are delayed, the consequences can extend beyond operational inconvenience, leading to financial losses and disruptions across institutions and markets.

In response to these challenges, the European Union introduced the Central Securities Depositories Regulation (CSDR) in 2014. As part of this framework, the Settlement Discipline Regime established a system of daily cash penalties applied to transactions that fail to settle on their intended date. Managing these penalties is a complex operational process. Financial institutions must process large volumes of data received from Central Securities Depositories, perform internal calculations, and reconcile the results with market data. When discrepancies arise, operational teams must investigate, apply corrections, and reprocess the affected penalties, all while maintaining full traceability and regulatory compliance.

VERMEG, a global fintech company, addresses these challenges through its MEGARA platform, a securities processing system that manages post-trade activities for financial institutions worldwide. Although MEGARA automates many aspects of penalty processing, the post-reconciliation correction phase still relies on manual intervention. When discrepancies occur between market data and internal calculations, operators must individually investigate each case, apply amendments, reset reconciliation indicators, and relaunch the reconciliation process. As penalty volumes grow, this manual workflow becomes increasingly difficult to manage and introduces risks in terms of errors and traceability.

This project was carried out during a work placement at VERMEG as part of the Master's program in Digital Management and Information Systems. Its objective is to analyze the existing correction workflow, identify its limitations, and design a more automated and reliable solution that improves efficiency, traceability, and compliance.

This report is organized into four chapters. The first introduces the general context, VERMEG, and the MEGARA platform. The second examines the technical and regulatory environment in detail. The third presents the design of the proposed solution. The fourth covers the implementation and testing phases.

Chapter One: General Overview

Introduction

This chapter presents the main elements required to understand the context, scope, and objectives of this Final Year Project. It begins with an introduction to VERMEG, its history, organizational structure, and product portfolio, followed by a presentation of the project context within the MegaCustody module of the MEGARA platform, including the existing CSDR penalty correction process, its limitations, and the proposed solution. It then describes the Agile working methodology adopted at VERMEG through Scrum and Kanban frameworks and concludes with an overview of the main tools and technologies used throughout the project.

The purpose of this chapter is to establish a clear understanding of the organizational, functional, and operational context of the project, and to explain why improving the post-reconciliation correction of CSDR penalties represents both an operational and strategic necessity. It serves as the foundation for the subsequent parts of the report, which will present a more detailed analysis of the platform, the regulatory framework, and the designed solution.

1. Presentation of VERMEG

1.1 Historical Overview

VERMEG is an international fintech company that develops software solutions for financial institutions. The company was originally founded in 1993 under the name BFI. In 2002, following a strategic evolution, the company was restructured and adopted the name VERMEG, as illustrated in Figure 1. Since then, it has followed a path of continuous growth driven both by organic development and by targeted acquisitions.

Over the years, VERMEG has expanded its activities to support several sectors of the financial industry, including banking, insurance, wealth management, and risk management. A major milestone occurred in 2014 when the company acquired the Belgian firm BSB, which specialized in financial software for insurance and private banking. This acquisition strengthened VERMEG's expertise in the insurance sector.

In 2018, VERMEG further expanded its capabilities through the acquisition of Lombard Risk, an international provider of software solutions focused on regulatory reporting, risk

management, and collateral management. This move significantly broadened the company's product portfolio and reinforced its position in regulatory technology.

Today, VERMEG operates globally with more than 1,700 employees across over 40 countries and serves more than 500 clients, primarily financial institutions. This international presence reflects the company's ability to adapt to evolving market requirements while continuously developing solutions that meet the needs of the financial industry [1].



Figure 1: VERMEG logo

1.2 Company Organization

VERMEG was founded by Badreddine Ouali and is currently led by its Chief Executive Officer, Tarek Achich, supported by an executive committee composed of experts in corporate strategy, information technology, banking systems, and financial services. Through its expertise and innovation-driven approach, VERMEG has established itself as a key provider of software solutions for financial institutions worldwide.

To better address the evolving needs of the financial sector, the organization is structured around three main strategic divisions: Corporate & Operations, Insurance, and Capital Markets, as shown in Figure 2. This organizational structure reflects the diversity of competencies within the company and enables an integrated approach to tackling technological, operational, and regulatory challenges in the financial industry.

This Final Year Project was carried out within the Capital Markets Operations Team under the supervision of Mr. Imed Ben Mimon. More specifically, the project was conducted within the APAC Team, led by Mme Azza Lamine, which focuses on supporting and managing operations related to clients in the Asia-Pacific region [1].

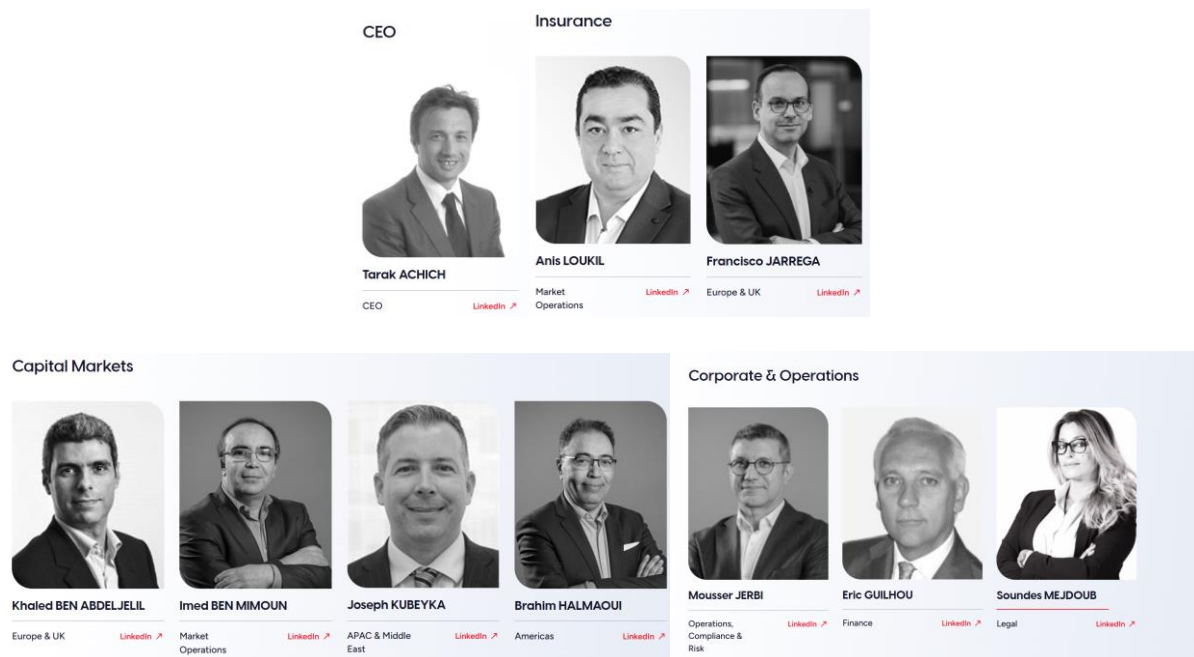


Figure 2: VERMEG Executive Committee Structure

1.3 VERMEG Product Portfolio

VERMEG provides a wide range of digital and software solutions designed to meet the needs of financial institutions such as banks, capital markets participants, and insurance companies. Its product portfolio is organized into three main categories: Banking Solutions, Insurance Solutions, and Digital Transformation Solutions.

MEGARA Securities Processing Platform

MEGARA is a global securities processing platform designed to automate middle office and back office operations for financial institutions, as illustrated in Figure 3. The platform is currently used by major banking institutions in more than twenty countries [1].



Figure 3: MEGARA Product logo

COLLINE Collateral Management Solution

COLLINE is a collateral management solution that supports a wide range of asset classes and provides automated calculations of risk exposure. Its logo is presented in Figure 4. The platform allows financial institutions to accurately evaluate collateral requirements and manage margin call processes efficiently [1].



Figure 4: Colline Product logo

Optimizer Collateral Optimization Platform

Optimizer is an intelligent platform designed to optimize collateral usage, unlock liquidity, and reduce funding costs. Its logo is presented in Figure 5. It digitizes collateral agreements, consolidates inventories, and applies advanced optimization strategies to automate collateral allocation decisions [1].



Figure 5: Optimizer Product logo

Solife Life Insurance and Pension Management

Solife is a comprehensive solution designed to manage life insurance, savings, pension, and provident contracts. Its logo is presented in Figure 6. It supports both individual and collective products while integrating tools that facilitate client relationship management and digital distribution channels [1].



Figure 6: Solife Product logo

Soliam Portfolio Management Platform

Soliam is VERMEG's integrated solution for portfolio management. Its logo is presented in Figure 7. The platform allows asset owners and wealth managers to manage the entire investment lifecycle within a single system.

Soliam supports multi-asset, multi-entity, and multi-currency operations while providing a complete front-to-back management environment. It covers processes ranging from order management and trade execution to settlement and reporting, giving institutions a comprehensive view of portfolio performance and risk exposure [1].



Figure 7: Soliam Platform logo

Veggo Financial Industry Low Code Platform

Veggo is a low-code platform designed specifically for the financial industry. Its logo is presented in Figure 8. It allows organizations to develop digital solutions faster by reducing the need for complex programming [1].



Figure 8: Veggo Platform logo

2. Context of the Project

This Final Year Project is conducted within the MegaCustody module of the MEGARA platform, which manages the clearing and settlement lifecycle of securities transactions. Within MegaCustody, a dedicated functionality known as Penalties Management handles CSDR settlement penalties throughout their complete lifecycle, from the reception of penalty reports sent by Central Securities Depositories (CSDs), through internal calculation and reconciliation with market data, to the final reporting and monthly closing.

2.1 Study of the Existing

The current penalty correction process within MEGARA follows an operational workflow that is initiated whenever reconciliation discrepancies occur.

Reconciliation is intended to ensure that the penalties calculated internally by MEGARA match the data received from the CSDs. When inconsistencies appear between these two sources of information, penalties remain in an unreconciled state and require investigation and correction before they can be processed again.

In such cases, the correction workflow involves several steps performed by system operators.

2.1.1 Current Workflow

The existing correction process typically involves the following sequence of actions:

Identification

Operators consult dedicated system screens to identify penalties that failed to reconcile. These interfaces allow users to detect discrepancies and analyze the penalties that require correction.

Amendment

Once problematic penalties have been identified, operators apply the necessary corrections through two types of amendments:

- Market Amendment, which corrects data received from the CSD.
- Client Amendment, which corrects internal system data used for penalty calculations.

Reconciliation Reset

After applying the amendments, operators reset the reconciliation attributes associated with the penalty. This includes clearing the reconciliation key, reconciliation status, and discrepancy indicators.

Reconciliation Job Execution

Once the reset is complete, a pre-configured trigger executes the reconciliation job according to its defined schedule, so that the corrected penalties go through the reconciliation process again using the updated data.

This workflow ensures that penalties can eventually reach a reconciled state through a combination of operator actions and scheduled job execution.

2.2 Problem

Although the current process can handle reconciliation discrepancies, it relies heavily on manual intervention, which creates several limitations in terms of efficiency, reliability, traceability, and scalability.

2.2.1 Operational Limitations

The manual workflow is time-consuming and difficult to scale. Processing around one hundred penalties can take several hours, as each case must be handled individually. During peak periods, such as month-end, this becomes even harder to manage, and teams spend too much time on repetitive tasks instead of analysis.

2.2.2 Reliability and Quality Limitations

Manual operations increase the risk of errors, such as incorrect selections or incomplete actions. The system also lacks validation checks to ensure that corrections are properly applied before reprocessing. In addition, the process depends on user experience, which can lead to inconsistent practices.

2.2.3 Traceability and Compliance Limitations

The current system does not provide a clear and centralized view of operations. Information is scattered across multiple logs, making it difficult to track the full lifecycle of a penalty. This lack of visibility complicates monitoring and can create challenges during audits.

2.2.4 Technical and Process Limitations

The correction process is not fully structured and depends on manual execution. Each step must be triggered separately, with no automated coordination between them. This makes the process harder to optimize and limits its ability to evolve.

2.3 Proposed Solution

To address these limitations, the project introduces a partially automated correction workflow designed to improve efficiency, reliability, and visibility.

2.3.1 Improving Operational Efficiency

To reduce processing time, the solution introduces batch processing and automated jobs that handle reconciliation resets. This significantly reduces manual effort and allows the system to manage higher volumes more efficiently.

2.3.2 Enhancing Reliability and Quality

To minimize errors, validation checks are added before reset and reprocessing operations. These ensure that only correctly amended penalties are processed, while a more structured workflow reduces inconsistencies between users.

2.3.3 Strengthening Traceability and Monitoring

To improve visibility, a monitoring dashboard provides a consolidated view of penalties, their status, and their progress. In addition, all key actions are clearly tracked, making it easier to audit and analyze the process.

2.3.4 Structuring the Process

The solution introduces a more organized workflow that connects all steps, from identification and amendment to reset and reprocessing. This reduces reliance on manual coordination and improves overall consistency.

2.3.5 Ensuring Data Integrity

Finally, the solution includes safeguards such as validation rules and controlled execution mechanisms. These prevent incorrect operations and ensure that all processing is consistent and reliable.

3. Working Methodology at VERMEG

In this section, we present the development methodology adopted at VERMEG for building and maintaining its software solutions. The discussion begins with an overview of the Agile methodology, which is the primary development approach used within the company. We then introduce two of its most widely used frameworks, Scrum and Kanban, which structure the daily work of development teams. Finally, a brief comparison with the traditional Waterfall methodology is provided to highlight the key differences between these development approaches.

3.1 Waterfall Methodology

The Waterfall methodology (Figure 9) is a traditional and structured approach to software development based on a linear and sequential process, where each phase must be completed before moving to the next. The development lifecycle typically includes requirement analysis, design, implementation, testing, deployment, and maintenance.

This model relies heavily on the accuracy of initial requirements, as changes are difficult to introduce once a phase is completed. Its main advantage lies in its clear structure, well-defined milestones, and strong documentation, making it suitable for projects with stable and predictable requirements.

However, its rigidity limits flexibility, as late changes may require revisiting previous phases, leading to delays and increased costs [2].

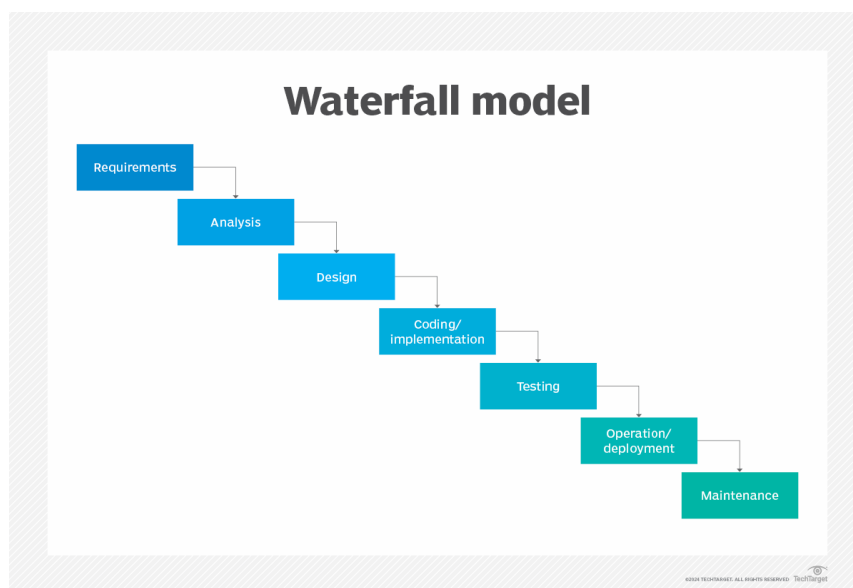


Figure 9: Waterfall Model

3.2 Agile Methodology

The Agile methodology (Figure10) [3] is based on principles that emphasize flexibility, collaboration, and continuous improvement. Unlike traditional models, Agile follows an iterative and incremental approach, allowing teams to deliver software progressively while adapting to changing requirements.

A key advantage of Agile is its ability to accommodate evolving needs throughout the development process. Through frequent releases and continuous feedback, stakeholders can evaluate progress and refine requirements at each stage. This adaptability enables organizations to respond quickly to market changes and client expectations.

Agile also promotes strong collaboration between developers, stakeholders, and product owners, ensuring better alignment with business objectives. In addition, it encourages continuous improvement through regular evaluations, helping teams optimize their processes and maintain high-quality standards.



Figure 10: Agile Methodology

3.3 Comparison Between Agile and Waterfall

Agile and Waterfall are two fundamentally different software development approaches, each suited to specific project contexts.

Waterfall follows a linear and sequential process where each phase is completed before the next begins. It is well adapted to projects with stable and clearly defined requirements, offering strong planning, documentation, and traceability.

In contrast, Agile adopts an iterative and incremental approach, delivering software in smaller increments with continuous stakeholder feedback. This flexibility allows teams to adapt to changing requirements and improve the product throughout the development process.

Overall, Waterfall provides structure and predictability, while Agile offers flexibility, faster delivery, and better responsiveness to change. A detailed comparison of these methodologies is presented in Table 1.

Table 1: Comparison between Agile and Waterfall Methodologies

Criteria	Agile	Waterfall
Approach	Iterative and incremental	Linear and sequential
Flexibility	Highly adaptable to changing requirements	Low flexibility; changes are difficult to introduce
Development Phases	Continuous iterations with feedback loops	Sequential phases executed one after another
Collaboration	Strong collaboration between teams and stakeholders	Limited collaboration after initial requirements
Time to Market	Faster delivery through incremental releases	Longer cycle with final delivery at project completion
Risk Management	Early detection of issues through continuous testing	Risks identified later in the development lifecycle
Documentation	Lightweight documentation focusing on working software	Extensive documentation defined upfront
Testing	Integrated throughout development	Performed after implementation phase
Best Suited For	Dynamic projects with evolving requirements	Projects with stable and predictable requirements

Both methodologies have their place depending on the context of the project. Waterfall remains relevant for projects where requirements are clearly defined and unlikely to change. Agile, on the other hand, is better suited for innovative and dynamic environments where flexibility, collaboration, and frequent delivery are essential.

3.4 Adapted Methodology at VERMEG

At VERMEG, Agile methodology is the preferred approach for software development due to its flexibility and adaptability in a fast-evolving financial environment. Unlike the rigid Waterfall model, Agile enables teams to work iteratively and continuously improve the system through regular feedback.

Financial software development involves complex regulations and changing market conditions. Agile allows teams to adapt functionalities as requirements evolve, ensuring alignment with both client needs and regulatory constraints.

Agile also enhances collaboration between developers, product managers, and stakeholders. Through short development cycles (sprints), teams deliver incremental improvements while continuously testing and validating features. This approach reduces risks, improves quality, and accelerates delivery.

At VERMEG, Agile practices are supported by key ceremonies such as daily stand-ups, sprint reviews, and retrospectives, which promote communication, transparency, and continuous improvement.

3.5 Adapted Methods

Within VERMEG's Agile environment, two complementary frameworks are primarily used to organize and manage development activities: Scrum and Kanban. These approaches provide structured yet flexible mechanisms that support collaboration, workflow management, and efficient delivery of software features.

3.5.1 Scrum Framework

Scrum is an iterative framework used to manage complex projects through short development cycles called sprints, typically lasting one to four weeks, as illustrated in (Figure 11). Each sprint aims to deliver a functional increment of the product.

Scrum defines key roles within the team: the Product Owner, who manages and prioritizes the backlog; the Scrum Master, who facilitates the process and removes obstacles; and the Development Team, which implements the product features.

The process is structured around several ceremonies, including Sprint Planning to define tasks, Daily Stand-ups to track progress, Sprint Reviews to present results and gather feedback, and Retrospectives to identify improvements.

Overall, Scrum promotes organization, transparency, and continuous improvement throughout the development process [4]

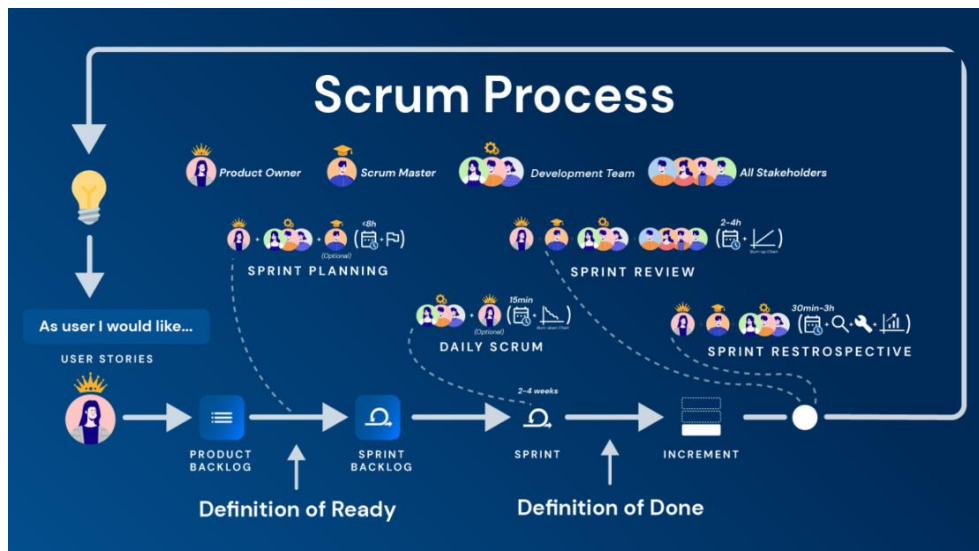


Figure 11: Scrum Process

3.5.2 Kanban Method

Kanban is a workflow management method focused on visualizing tasks and optimizing their flow, as illustrated in Figure 12. It enables teams to manage work in progress efficiently while maintaining a continuous and controlled workflow.

At VERMEG, Kanban is primarily used for operational activities such as support tasks, bug fixing, and incremental improvements. Unlike Scrum, which relies on fixed sprints, Kanban allows continuous task management without predefined time cycles.

Its key practices include visualizing tasks through Kanban boards, limiting work in progress to avoid bottlenecks, monitoring workflow to improve efficiency, and promoting continuous improvement.

By combining Scrum and Kanban, VERMEG adopts a hybrid Agile approach that balances structured development with flexible task management, ensuring both efficiency and adaptability [5].

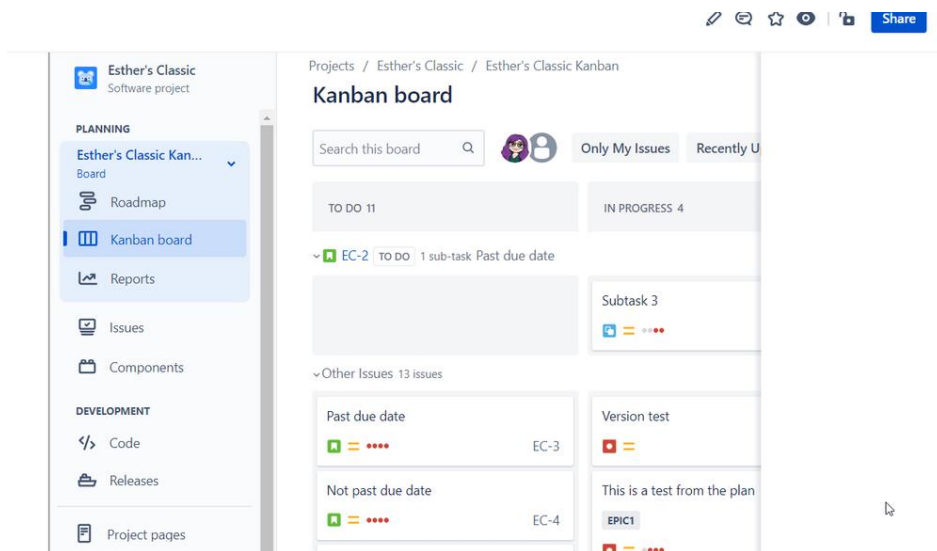


Figure 12: Kanban Board In Jira

4 Technologies Used

Throughout the project, several tools and technologies were used to support planning, collaboration, development, and system management. These tools facilitate communication, project tracking, documentation, and technical operations within the development environment.

4.1 Project Management and Collaboration Tools

JIRA is widely used as the main Agile project management tool, as presented in Figure 13. It allows teams to track user stories, defects, tasks, and sprint progress while organizing development activities efficiently [5].



Figure 13: Jira logo

Confluence (Figure 14) is used as a documentation and knowledge-sharing platform where project documentation, technical specifications, and development notes are maintained and updated collaboratively [5].



Figure 14: Confluence logo

Microsoft Teams serves (Figure 15) as the primary communication platform for meetings, instant messaging, file sharing, and collaboration between team members [6].



Figure 15: Microsoft Teams Serves logo

Office 365 (Figure 16) provides productivity tools such as Word, Excel, PowerPoint, and Outlook, enabling teams to create documents, analyze data, and collaborate in real time [6] .



Figure 16: Microsoft Office 365

MyStandards

MyStandards (Figure 17) is a SWIFT platform that helps financial institutions manage and implement messaging standards efficiently. It is primarily used to work with SWIFT formats such as MX (XML-based) and MT messages.

The platform allows users to explore message structures, including fields and tags, helping them better understand and apply SWIFT standards in financial communication [7].

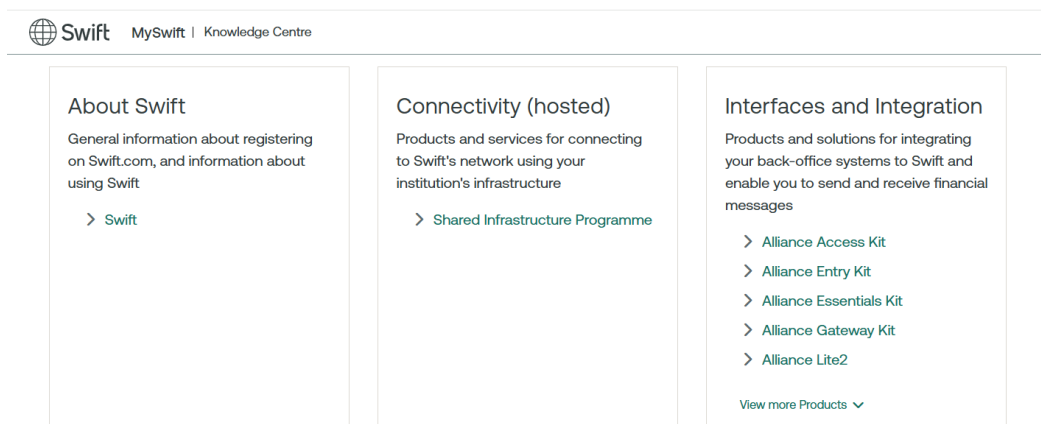


Figure 17: Screenshot of MyStandards page

4.2 Financial Software Platform

MEGARA

MEGARA is VERMEG's securities processing platform used to automate post-trade operations and manage CSDR penalties through the MegaCustody module; further details are provided in Chapter 2.

4.3 Automated Testing Tools

Qualimetry

Qualimetry is an internal testing tool used at VERMEG to automate system validation and ensure software quality. It uses SQL queries to compare expected results with actual system data, allowing quick detection of errors and inconsistencies. By reducing manual checks, it makes testing faster, more reliable, and easier to repeat.

4.4 API Testing Tool

SoapUI

SoapUI (**Figure 18**) is a tool used to test APIs and web services. It allows testers to send requests, analyze responses, and verify that APIs behave as expected. By supporting both SOAP and REST services, it helps ensure reliable integration between MEGARA and external systems [8].



Figure 18: SoapUi logo

4.5 File Transfer and Security Tools

WinSCP

WinSCP (**Figure 19**) is a tool used for secure file transfer between local machines and remote servers. It supports protocols like SFTP and SCP, ensuring safe data exchange. In this project, it was mainly used to transfer files, logs, and configuration data during testing and operations [9].



Figure 19: WinSCP logo

Conclusion

This chapter presented the general context of the project, including VERMEG's product ecosystem, the role of the MEGARA platform, and the operational challenges surrounding the post-reconciliation correction of CSDR penalties within the MegaCustody module. It established the main problem this project aims to address and introduced the general direction of the proposed solution. The following chapter will examine in greater detail the MEGARA platform, the CSDR regulatory framework, and the complete penalty processing lifecycle.

CHAPTER 2: Product Overview and Business Context

Introduction

This chapter explores two key elements essential to understanding the solution developed in this project: the Megara platform and the CSDR regulatory framework governing settlement penalties. It presents the architecture of the Penalties module, explains the penalty calculation rules, and details the complete daily penalties workflow within Megara, from data reception through reconciliation and monthly closing. This analysis forms the foundation for the solution design presented in Chapter 3.

1. The Megara Platform

1.1 What Megara Does

1.3.1 Overview of MEGARA

MEGARA is a securities processing platform that has evolved over more than two decades to adapt to regulatory developments and best practices in financial markets. The platform is designed to optimize Straight Through Processing and automate post-trade operations through a flexible and modular architecture.

One of the key characteristics of MEGARA is its three-view architecture. The **Operations View** manages internal processing logic and operational workflows. The **Client View** focuses on services provided to clients, including reporting and client-level data management. The **Market View** handles interactions with external market infrastructures such as Central Securities Depositories and messaging networks (Figure20) [1].

This architecture allows financial institutions to integrate new markets, instruments, and regulatory requirements without disrupting existing operations. As a result, MEGARA provides a scalable and adaptable environment capable of supporting complex post-trade processes for a wide variety of financial institution clients (Figure 21) [1]



Figure 20: High-Level Functional Architecture of the Megara Platform



Figure 21: MEGARA's Global Client Base

1.3.2. MEGARA Applications and Modules

MEGARA is composed of several business modules (Figure 23) [1]that can be deployed individually or integrated into a comprehensive platform. All modules share a common core infrastructure that includes static data management, position keeping, accounting functionalities, and connectivity with external systems such as SWIFT and Central Securities Depositories.

The main modules of the platform include:

- **MegaCommon** : which centralizes reference data used across all MEGARA components.
- **MegaCustody**: which manages clearing and settlement processes from instruction creation to final settlement and **CSDR Penalties Management**
- **MegaCor** : which handles corporate actions processing from event notification to execution.
- **MegaBroker** : which centralizes incoming and outgoing messaging flows across the platform.
- **MegaCM**: which manages collateral and exposure operations for central banks.
- **MegaCash**: which processes cash payments generated by custody operations.
- **MegaCompliance**: which ensures that investment funds comply with regulatory requirements.

And Within the MegaCustody module, a dedicated functionality called **Penalties Management** handles the lifecycle of CSDR penalties. This functionality plays an important role in the context of this project, as it manages the reception of penalty reports, the internal calculation of penalties, reconciliation with market data, and the generation of reports related to settlement penalties.

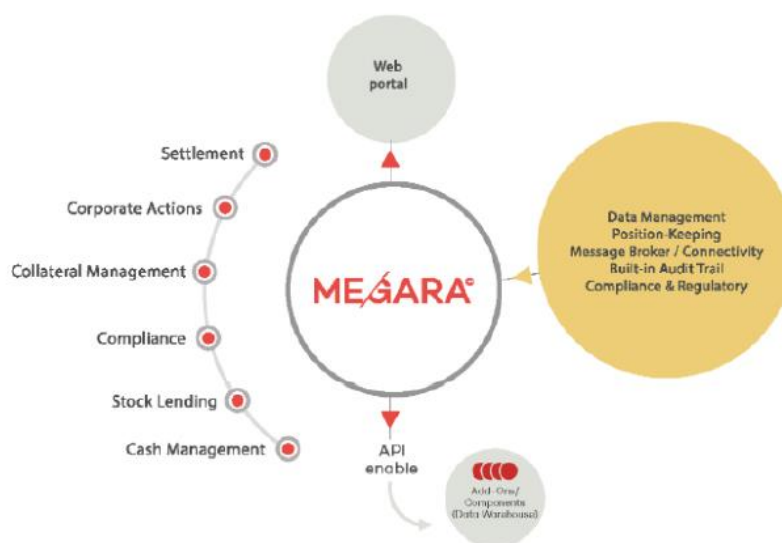


Figure 22: MEGARA Suite Applications

1.2 Penalties Functionality in the Settlement Module

Within Megara, the functionality related to settlement penalties is integrated into the **Settlement module**. It was designed to ensure compliance with the CSDR Settlement Discipline Regime and supports the complete lifecycle of settlement penalties, from their reception to reconciliation, invoicing, and reporting.

According to VERMEG's internal documentation, the system manages two types of cash penalties defined by the regulation:

● **Settlement Fail Penalty (SEFP)**: applied daily to any matched settlement instruction that fails to settle on its intended settlement date and remains pending.
Late Matching Fail Penalty (LMFP): applied when a settlement instruction is matched after the relevant settlement cut-off time. In this case, the penalty is charged to the counterparty that was last to enter or modify the instruction.

Penalties can enter the system through two main channels. The first is automatic: Megara receives penalty messages from the CSD, either through **SWIFT MT537 or MT548 messages in ISO 15022 format**, or through **semt.044 messages in ISO 20022 format**, which are then parsed and integrated into the penalties database. The second channel is manual: operators can use a contingency interface to simulate the reception of a penalty. This capability is particularly useful for testing scenarios or for managing exceptional cases where a CSD message was not received correctly.

Once penalties are registered in the system, the platform manages their lifecycle through several processing phases, including allocation (especially when global instructions must be distributed to child instructions), amendment management, repair of incomplete penalties, reconciliation, invoicing, and finally the generation of statements for end clients.

1.3 Penalty Lifecycle Phases

To clearly understand the origin of the problem addressed in this project, it is necessary to examine the complete lifecycle of a penalty within MEGARA, as illustrated in **Table 2**.

Table 2: Penalty Lifecycle within MEGARA

Phase	Title	Core Activities	Output / Data State
01	Reception	Parsing SWIFT/ISO files, Business/Technical validation, De-duplication.	Validated Raw Penalty (DAILY_PENALTY table)
02	Enrichment	Augmenting data with Security Master, FX rates, and Counterparty info.	Enriched Penalty Record
03	Internal Calculation	Independent application of CSDR rules to find "Client Amount."	Calculated Client Amount
04	Allocation	Distributing aggregated penalties to underlying child instructions.	Allocated Penalty Items
05	Reconciliation	4-way check: Amount consistency, Reference data, Status, and Existence.	Reconciliation Status (Matched/Unmatched)
06	Amendment	Critical Phase: Manual investigation and correction of discrepancies.	Corrected/Verified Record
07	Invoicing	Creation, validation, and payment simulation of client bills.	Final Invoice
08	Statement	Generation of MT537 messages for settlement fail overviews.	Client Penalty Statement

2. The CSDR Regulatory Framework

2.1 The Settlement Discipline Regime

The Central Securities Depositories Regulation (CSDR) introduced the **Settlement Discipline Regime (SDR)** to address a persistent issue in European financial markets: a high number of settlement instructions were failing without any significant consequences for the parties responsible.

Before the introduction of the SDR, which was gradually implemented between 2020 and 2022, a settlement fail was considered operationally problematic but did not automatically

trigger a financial penalty. As a result, market participants had limited incentives to prioritize settlement efficiency.

The SDR changed this situation by introducing two main mechanisms designed to improve settlement discipline:

First, **cash penalties**, which are daily financial penalties applied to settlement instructions that fail to settle on their intended settlement date.

Second, **mandatory buy-ins**, a mechanism allowing the affected party to force the purchase of the undelivered securities in the market after a prolonged settlement failure.

This project focuses exclusively on the first mechanism: **cash penalties and the way they are processed within the Megara platform**.

2.2 How Penalties Are Calculated

The CSDR penalty calculation is defined in the ESMA Regulatory Technical Standards (RTS) and follows a relatively straightforward formula:

$$\text{Daily Penalty} = \text{Reference Amount} \times \text{Penalty Rate} \times (1 / 360)$$

Where:

- **Reference Amount** corresponds to the undelivered quantity multiplied by the reference price of the security.
- **Penalty Rate** depends on the liquidity category of the security.
- **1/360** represents the day-count convention used to convert the annual rate into a daily penalty.

The penalty rate varies depending on the liquidity of the instrument, which is determined according to criteria such as trading volume and market activity. The main penalty rates defined under CSDR are summarized in **Table 3** [10].

Table 3: CSDR Penalty Rates by Instrument Category

Instrument Category	Penalty Rate (bps/year)	Equivalent Daily Rate
Liquid shares	100 bp (1.00%)	~0.0028% per day
Less liquid shares	50 bp (0.50%)	~0.0014% per day
Liquid government bonds	25 bp (0.25%)	~0.0007% per day
Less liquid government bonds	12.5 bp (0.125%)	~0.00035% per day
Other debt instruments	Variable by liquidity	—

To illustrate this mechanism, consider a transaction involving **10,000 shares of a liquid equity priced at €50 per share**. The reference amount is therefore €500,000. With a penalty rate of **1.00 percent per year**, the daily penalty is calculated as:

$$€500,000 \times 1\% \times (1 / 360) = €13.89$$

If the settlement instruction remains unsettled for five days, the total penalty would reach **€69.45**.

An important characteristic of the CSDR penalty mechanism is its **symmetrical nature**. The party responsible for the settlement failure pays the penalty, while the counterparty affected by the failure receives the corresponding amount. The CSD acts as a neutral intermediary responsible for calculating, collecting, and redistributing these penalties.

2.2.1 The Role of CSDs

Under the CSDR framework, **Central Securities Depositories (CSDs)** are responsible for managing the penalty process at the market level. Each business day, they identify settlement instructions that have failed, calculate the corresponding penalties based on the regulatory rules, and transmit the results to their market participants.

In addition, CSDs collect penalties from the paying participants and redistribute them to the receiving participants.

From Megara's perspective, this means the platform must receive and process two main categories of penalty reports from CSDs.

The first category consists of **daily reports**, transmitted on the business day following the relevant settlement date. These reports contain detailed information about individual

penalties associated with each failed settlement instruction. They are typically transmitted via **MT537 or MT548 messages in ISO 15022 format**, or through **semt.044 messages in ISO 20022 format**.

The second category consists of **monthly reports**, which aggregate all daily penalties accumulated over the calendar month. These reports serve as the basis for the final financial settlement between market participants.

One of Megara's key responsibilities is to reconcile these two reporting layers, ensuring consistency between the **daily penalties received during the month** and the **final monthly totals** communicated by the CSD. This reconciliation relies on the **Penalty Common Reference tag (20C: PCOM)**, which uniquely identifies each penalty and is shared by both the paying and receiving participants.

2.2.2 Compliance and Reporting Obligations

Beyond the operational handling of penalties, financial institutions must also comply with a range of regulatory and internal control obligations.

Institutions are required to maintain internal processes that verify the accuracy of penalties received from CSDs, ensure that penalties are correctly recorded in their accounting systems as either charges or revenues depending on their position, and provide clear audit trails demonstrating that the penalty management process has been correctly followed.

In some cases, institutions must also comply with **regulatory reporting requirements**, such as submitting statistics related to settlement failure rates and associated penalties to national supervisory authorities.

These obligations highlight the importance of having a **transparent, traceable, and reliable penalty management process**. A robust system is therefore not only operationally beneficial but also essential for meeting regulatory compliance requirements

3 Daily Penalty Processing in Megara

3.1 Receiving and Loading Penalties

Each business day, after the settlement cut-off time, Central Securities Depositories generate daily penalty reports and transmit them to their participants. Megara retrieves these reports automatically, either from the SWIFT network or from a predefined file directory, and processes them through a structured loading workflow.

This workflow includes several steps. The system first parses the incoming message to interpret its format and structure. It then performs technical validation to ensure the message is complete and conforms to the expected format. After that, business-level validations are applied to verify the correctness of key elements such as the ISIN, the coherence of dates, and the validity of penalty amounts. Finally, the system performs a deduplication check to ensure that the same message is not processed more than once.

If a message fails any of these validations, it is automatically redirected to an exception queue and an alert is generated for the operations team. This mechanism prevents incorrect or incomplete data from entering the downstream processing stages. Messages that successfully pass all validations are stored in the **DAILY_PENALTY** table and marked with a status indicating that they are ready for the next step of processing.

Each penalty record contained in these messages includes all the information required for further processing. This typically includes a penalty reference, the identifier of the related settlement instruction, the ISIN of the security, the undelivered quantity, the reference price, the applicable penalty rate, the calculated penalty amount, the value date, the counterparty involved, and the indication of whether the amount represents a debit or a credit for the participant.

3.2 Enrichment and Internal Calculation

After the penalties have been loaded into the system, Megara performs an enrichment process by linking the raw penalty data with information stored in its internal repositories. This step supplements the penalty records with additional contextual data that is necessary for accurate processing.

The enrichment process includes retrieving security classification data used to determine the applicable penalty rate, counterparty information, reference prices, and exchange rates

when relevant. It also associates the penalty with the corresponding settlement instruction recorded in the system.

Once this additional data is available, Megara performs its own internal calculation of the penalty amount using the same formula defined by the CSDR regulation. The result of this calculation is stored as the **client amount**, which represents the penalty amount calculated internally by the system. This value is later compared with the **market amount** provided by the CSD.

Having both a market amount and a client amount allows Megara to perform meaningful reconciliation. Any discrepancy between these two values indicates that further investigation may be required.

The calculation logic also handles several operational edge cases. For example, if the reference price of a security is missing, the system may fall back to the price provided by the CSD or flag the record for manual input. In cases of partial deliveries, only the undelivered quantity is considered when calculating the penalty. The system also accounts for holiday adjustments in accordance with the rules defined in the Regulatory Technical Standards.

3.3 Reconciliation

The reconciliation process compares the penalty information received from the CSD with the penalty amount calculated internally by Megara. This comparison ensures that both sources produce consistent results and helps identify discrepancies that require investigation.

The reconciliation engine evaluates four main dimensions.

The first dimension is **amount matching**, which checks whether the market amount and the client amount are sufficiently close. A configurable tolerance threshold, such as one cent, is typically used to accommodate minor rounding differences.

The second dimension is **reference matching**, which verifies that key attributes such as the ISIN, settlement date, quantity, and counterparty are consistent between the two records.

The third dimension is **status matching**, which confirms that both sides agree on which participant should be debited and which participant should be credited.

The fourth dimension is **existence matching**, which ensures that a corresponding settlement instruction exists in Megara for every penalty received from the CSD.

Based on these checks, each penalty is assigned a reconciliation status. Examples of such statuses include Totally Reconciled, Partially Reconciled, Not Reconciled, Amount Mismatch, Reference Mismatch, or Status Mismatch. When two records are successfully matched, they are linked through a reconciliation key known as **RECONCILIATION_PK**.

Reconciliation is performed on a daily basis, but it also occurs at the monthly level. At the end of each month, Megara verifies that the cumulative total of all daily penalties matches the monthly summary report received from the CSD. This monthly reconciliation relies on the **Penalty Common Reference**, represented by the **20C:PCOM tag** in SWIFT messages, which uniquely identifies each penalty for both the failing participant and the receiving participant.

4 Handling Discrepancies: Amendments and Corrections

4.1 Why Discrepancies Occur

In real operational environments, not all penalties reconcile successfully during the first reconciliation cycle. Discrepancies can arise for several reasons related to differences between the data received from the CSD and the data stored or calculated within the participant's internal system.

One common cause is **amount mismatches**. These occur when the CSD and the client calculate the penalty using slightly different parameters. For example, they may rely on different reference prices, apply different rounding conventions, or classify the security under different liquidity categories, which leads to different penalty rates.

Another source of discrepancy is **reference mismatches**. These appear when key attributes do not match between the two sides of the comparison. Examples include an incorrect ISIN, an incorrect counterparty identifier, or a settlement date that differs between the CSD report and the internal settlement instruction.

A third category involves **status mismatches**. In these cases, the two sides disagree about which party is responsible for the settlement failure. Such situations may arise when settlement instructions are amended after the intended settlement date, which can create ambiguity regarding the party considered to be in default.

Finally, **existence mismatches** can occur when a penalty is reported for a settlement instruction that does not exist in Megara, or when Megara calculates a penalty internally for

an instruction that was not penalized by the CSD. These situations require additional investigation before reconciliation can be completed.

4.2 The Amendment Mechanism

When a discrepancy is detected, the corresponding data must be corrected before the penalty can be reconciled again. Megara provides two types of amendments, each corresponding to one side of the reconciliation comparison.

A Market Amendment is used to correct the data received from the CSD. This may involve adjusting an incorrect penalty amount, correcting an ISIN, or updating an erroneous quantity. In some situations, the correction occurs because the CSD itself sends a follow-up message correcting the original information. This typically takes the form of a Forward Amendment (FWAM) message. In other situations, the operator may identify the issue and apply a manual correction while waiting for the official amendment from the CSD.

A Client Amendment, on the other hand, is used to correct internal data within the participant's system. For example, an operator may need to update the quantity or settlement date of a settlement instruction, or correct the reference price used for the internal calculation. When a Client Amendment is applied, Megara automatically recalculates the corresponding client penalty amount to reflect the updated data.

There are also cases where both types of amendments must be applied together. For instance, if a client corrects the quantity of a settlement instruction internally, a Client Amendment is triggered. However, if the CSD has not yet received or reflected this change, the market-side data may also need to be adjusted through a Market Amendment. Coordinating these combined corrections can make the process more complex for operations teams.

To ensure traceability, every amendment, whether market-side or client-side, is recorded in the **AMENDMENT_HISTORY** table. Each record stores the previous and updated values, the identity of the operator who performed the modification, and the exact timestamp of the change. This provides an audit trail for the amendment process, although the current workflow still presents certain limitations in terms of operational efficiency and traceability.

4.3 Penalty Repair

In addition to amendments, Megara also supports a process known as **penalty repair**. This mechanism is used for penalties that were initially created with an **INCOMPLETE** status because they violated one or more business rules at the time of their creation.

Such situations can occur when the system cannot identify a corresponding market instruction, or when the associated settlement instruction is marked as ineligible for penalties according to internal rules.

To address these cases, Megara provides a dedicated repair interface. Through this interface, the operator can select the incomplete penalty, identify the correct market instruction, and initiate the repair operation. Once the underlying business rule violation is resolved, the penalty status is updated and the record re-enters the standard processing workflow, allowing it to proceed through reconciliation and the subsequent processing stages.

4.5 Post-Reconciliation Correction: The Current Process and Its Limits

4.5.1 Why a Reset Is Needed After Amendment

When an amendment is applied, the underlying data of the penalty is corrected. However, the penalty record still retains the reconciliation attributes assigned during the previous reconciliation run. This means that it still carries the reconciliation status it previously received, such as *Amount Mismatch* or *Not Reconciled*. It also remains linked to the original reconciliation key stored in **RECONCILIATION_PK**, and the discrepancy values calculated during the earlier reconciliation attempt remain recorded in the system.

If the reconciliation process were executed again without first clearing these attributes, the system could produce inconsistent results. In some cases, the penalty might be skipped because it is considered already reconciled. In other cases, the system could combine outdated reconciliation information with newly corrected data, leading to incorrect outcomes.

For this reason, a **reconciliation reset** is required before running reconciliation again. The reset operation clears the existing reconciliation status, removes the reconciliation key, and deletes the discrepancy values generated during the previous reconciliation attempt. This ensures that the penalty can go through a clean reconciliation cycle based solely on the corrected data.

4.5.2 How the Reset Currently Works

In the current implementation, the reset process is entirely manual and requires several steps performed by an operator.

First, the operator must access the **post-reconciliation correction interface**, which is restricted to authorized users due to the sensitive nature of the operation.

Next, the operator must identify all penalties that have been amended and require a reconciliation reset. Each relevant penalty must be selected manually, ensuring that none are missed and that penalties not requiring a reset are not included.

Once the correct penalties have been selected, the operator activates the **Reset Reconciliation** function. This operation clears the reconciliation attributes by setting **RECONCILIATION_PK** to null, removing the **RECONCILIATION_STATUS**, and deleting the discrepancy amounts stored during the previous reconciliation run. These changes are applied through a single database transaction.

After the reset is completed, the operator must manually trigger the reconciliation job using the system's job scheduler. The operator defines the scope of the reconciliation run, waits for the process to complete, and then verifies the resulting reconciliation statuses.

If the amendment resolved the discrepancy correctly, the penalties should now appear as **Totally Reconciled**. If not, the operator must begin another cycle of investigation and correction.

4.5.3 Where the Process Falls Short

Because the entire reset and re-reconciliation procedure depends on manual actions, several operational challenges arise.

From an **operational perspective**, the process is time-consuming and difficult to scale. Each penalty requires multiple actions across different screens, and there is no automated mechanism to streamline the workflow. During periods of high activity such as month-end processing, market disruptions, or technical incidents at CSDs, the volume of penalties can quickly exceed what operations teams can realistically manage within a limited timeframe.

From a **reliability perspective**, the process depends heavily on the accuracy and attention of the operator. The operator must remember which penalties were amended, select the

correct records for reset, and execute the sequence of actions in the proper order. The system currently does not enforce safeguards to verify that a penalty was actually amended before it is reset. It also does not ensure that both sides of a combined amendment have been completed before reconciliation is restarted. Furthermore, an operator may relaunch the reconciliation job even if not all necessary resets have been performed.

From a **traceability perspective**, the process presents additional limitations. The various actions involved in amendments, resets, and reconciliation runs are recorded in separate logs and database tables that were not designed to be analyzed together easily. As a result, reconstructing the complete history of a single penalty can require examining multiple data sources. For example, determining how many times a penalty was amended, when it was reset, and why it continued to fail reconciliation may involve manual investigation across several tables. This lack of consolidated traceability can complicate internal reviews and external audits.

These challenges combine to create three main issues: operational inefficiency, potential reliability risks, and limited traceability. Addressing these issues is the central objective of this project, and the following chapter presents the proposed solution.

4.6 Monthly Aggregation and Downstream Effects

An important characteristic of Megara's architecture is that **monthly penalties are dynamically derived from their underlying daily penalties**. Monthly penalty values are not stored independently; instead, they are calculated by aggregating the daily penalties associated with the same period.

As a result, when the reconciliation status of a daily penalty changes, the corresponding monthly penalty is automatically recalculated. For example, if a daily penalty moves from *Amount Mismatch* to *Totally Reconciled* after a correction and reconciliation cycle, the monthly totals that include this penalty will be updated accordingly.

This relationship means that the accuracy of daily reconciliation directly influences the correctness of monthly penalty amounts. If a daily discrepancy remains unresolved at the end of the month, it will affect the monthly aggregation and therefore distort the final penalty totals used for financial settlement.

These monthly totals are used as the basis for **client invoicing and financial settlement between market participants**. Consequently, errors or unresolved discrepancies at the daily level may propagate into downstream processes, potentially leading to incorrect charges or credits for clients.

Because of this downstream impact, the correction process must not only be efficient but also reliable and complete. Any mistake during the reset or re-reconciliation phase can propagate into invoicing, reporting, and financial settlement activities.

Conclusion

This chapter examined the Megara platform and the CSDR regulatory framework that forms the basis of this project. It described the complete penalty lifecycle within Megara, from data reception through reconciliation and client reporting, and identified the post-reconciliation correction phase as the stage that still relies on manual intervention. The following chapter presents the design of the solution developed to address this.

CHAPTER 3: Solution Design and Development

Introduction

The previous two chapters established the foundation of this project. Chapter 1 introduced VERMEG and the Megara platform, while Chapter 2 detailed the CSDR regulatory framework and the limitations of the current post-reconciliation correction process. What emerged from that analysis was a well-defined problem: the manual operations required to reset and re-reconcile unmatched penalties are too slow, too fragile, and too difficult to audit at scale.

This chapter presents the response to that problem. It walks through the design of the solution from the ground up, starting with a careful reading of what the business actually needs, moving through the architectural choices that shaped the implementation, and closing with UML models that describe the system formally. The focus throughout is on two batch jobs: the Reconciliation Reset Job and the Notification Reset Job.

3.1 Needs Analysis and Constraints

3.1.1 Business Needs

Requirements that are truly useful do not come from reading specifications in isolation. They come from watching how people work and paying attention to where the friction is. The analysis phase involved close collaboration with operations teams at VERMEG and with regular users of the Megara platform. Five core needs emerged consistently from those conversations.

Need 1: An automated reset process

Resetting a penalty's reconciliation attributes today involves a manual sequence of steps with no system-enforced order and no safeguards. The operator navigates to a specific screen, locates the right records, clears the right fields, and then manually restarts the reconciliation job. There is nothing to prevent a step being skipped, and there is no consistent log of what was done.

What the operations team needs is a batch job that handles this reliably. Not a guided wizard or a complex workflow screen, but a solid, predictable job that can be triggered when

needed, processes only the records it should, and leaves a full trace of every operation it performs.

Need 2: Independent handling of notification resets

Reconciliation resets and notification resets address different fields in the database and arise from different operational situations. Coupling them into a single job would force unnecessary operations in cases where only one type of reset is needed, and would make troubleshooting harder when something goes wrong.

The team needs two independent jobs, each with its own schedule and its own execution log. Triggering one should have absolutely no effect on the other.

Need 3: Controlled access and safe execution

Resetting reconciliation data is a sensitive operation. Applied to the wrong set of records, it can corrupt data that was correctly matched. The current process has no guardrails: access controls are minimal and there are no pre-execution checks to verify that the targeted records are actually eligible.

The solution needs to restrict manual triggering to authorized users, verify the eligibility of every record before touching it, and wrap the entire batch in a single database transaction. If anything fails partway through, the whole operation rolls back with nothing left in an inconsistent state.

Need 4: A reliable audit trail

CSDR compliance is not only about producing correct numbers. It also requires being able to demonstrate, at any point, exactly what happened to a given penalty and when. Today, reconstructing that history means piecing together information from several different tables and log files with no consistent structure between them.

Every execution of a reset job, whether triggered manually by an operator or automatically by the scheduler, must produce a structured log entry. That entry should record the execution mode, the timestamp, the number of records processed, and the outcome. The log must be persistent and available for audit at any time.

Need 5: No disruption to existing functionality

Introducing changes to a production financial platform always carries risk. The solution must leave the penalty calculation engine completely untouched, must not write directly to the monthly penalty tables, and must not cause any regression in the parts of Megara that already work correctly. Integration with the existing scheduler, security framework, and data access layer is mandatory.

3.1.2 Design Constraints

Alongside the business needs, the design must respect a set of hard constraints arising from CSDR regulation and from the technical and organizational standards at VERMEG. These constraints represent firm boundaries, as summarized in **Table 4**.

Table 4: Key Regulatory, Technical, and Organizational Constraints

Ref	Constraint	Implication for the solution
C1	CSDR compliance: the penalty calculation rules defined by ESMA's RTS must not be modified under any circumstances.	The jobs operate exclusively on reconciliation and notification attributes. The calculation engine is out of scope.
C2	Data integrity: market and client data may only be modified through the existing amendment mechanism.	The reset jobs introduce no new modification pathway. They clear specific fields that are designed to be clearable.
C3	Monthly penalties result solely from the automatic aggregation of daily records and must never be modified directly.	The jobs have no access to the monthly penalty tables.
C4	Megara architecture: the solution must use existing frameworks, batch patterns, and the native scheduler.	Both jobs are registered in Megara's scheduler and follow the same implementation conventions as existing batch jobs.
C5	Access control: sensitive operations must be restricted to users with the appropriate role.	Manual triggering requires a specific Megara role. Enforcement is handled through the existing RBAC mechanism.
C6	Transactional atomicity: every job execution must be all-or-nothing, with automatic rollback on failure.	All database operations are wrapped in a single transaction per execution. Partial updates are structurally prevented.

3.2 Functional Objectives

3.2.1 Primary Objective

The primary objective is to automate the post-reconciliation correction process through two independent batch jobs that are safe, fully traceable, and consistent with how Megara already operates. The correct operation should be straightforward to perform; anything unsafe should be structurally impossible.

3.2.2 Secondary Objectives

- Automate the Reconciliation Reset Job: build a job that clears RECONCILIATION_PK, RECONCILIATION_STATUS, and mismatch amounts for all amended and non-reconciled daily penalties. It must support both manual triggering by an authorized operator and scheduled automatic execution.
- Automate the Notification Reset Job: build a separate and fully independent job that clears notification attributes for penalties that were already notified and later corrected. It carries the same dual-trigger requirement as the first job, with its own independently configurable schedule.
- Maintain strict independence between the two jobs: each job has its own schedule, its own eligibility logic, its own execution log, and no awareness of the other job's state or history.
- Guarantee safe execution at all times: atomic transactions with automatic rollback, pre-execution eligibility checks, role-based access control, and concurrency protection to prevent two instances from processing the same records at the same time.
- Ensure complete and durable traceability: every execution, whether manual or scheduled and whether it succeeds or fails, must produce a persistent structured log entry. This is a regulatory compliance requirement.

3.3 Solution Architecture

3.3.1 The Two Components

The solution consists of two batch jobs that extend Megara's existing batch infrastructure without requiring any changes to components that are already in production. A detailed description of these components is provided in **Table 5**.

Table 5: Description of the Two Batch Components

Component	Type	Responsibility	Trigger mode
Reconciliation Reset Job	Batch Job	Clears RECONCILIATION_PK, RECONCILIATION_STATUS, and mismatch amounts for eligible amended penalties. Runs atomically with full rollback protection.	Manual (authorized operator) or scheduled (Megara scheduler)
Notification Reset Job	Batch Job	Clears notification attributes for notified and corrected penalties. Fully independent from the Reconciliation Reset Job in every respect.	Manual (authorized operator) or scheduled (independently configurable)

3.3.2 Design Principles

Five principles shaped the design decisions made in this project.

Principle 1: Fit naturally into Megara's existing patterns

Both jobs use the same frameworks, coding conventions, and infrastructure as every other batch job in Megara. There is no bespoke engine, no new dependency, no unfamiliar technology. Any VERMEG engineer who knows the platform can read, maintain, and extend this code without any project-specific onboarding.

Principle 2: Transactions without exception

Every job execution wraps all its database writes inside a single transaction. If the job is processing three hundred penalties and encounters a failure on the two hundred and forty-seventh, the entire batch rolls back automatically. The database is either fully updated or left completely unchanged. There is no scenario in which a penalty ends up in a partial or inconsistent state.

Principle 3: Strict separation of concerns

The two jobs have separate codebases, separate schedules, separate execution logs, and separate eligibility criteria. Neither job has any knowledge of the other. This makes each one independently testable, independently deployable, and straightforward to diagnose when something needs investigating.

Principle 4: Reuse existing security infrastructure

The solution extends the role-based access control mechanism already present in Megara rather than introducing a new one. The same roles that govern other sensitive batch operations are extended to cover job triggering. No new security surface area is created.

Principle 5: Observability built into the execution

Every execution writes a structured log entry to the JOB_EXECUTION_LOG table as a mandatory step, not as optional instrumentation. This output is the primary artifact used to demonstrate CSDR audit compliance and to diagnose any unexpected behavior after a run.

3.3.3 Layered Architecture

The solution respects Megara's layered architecture. Each layer has a clearly defined scope, and the boundaries between them are maintained, as illustrated in **Table 6**.

Table 6: Megara Layered Architecture and Responsibilities

Layer	Components	Responsibility
Presentation	Manual job trigger controls in the existing Megara interface	Capture operator input and enforce role-based visibility of trigger controls
Business Logic	ReconResetJob, NotifResetJob, eligibility validation	Apply business rules, manage transactions, enforce access control, coordinate audit logging
Data Access	PenaltyDAO, JobExecutionLogDAO	Execute optimized SQL queries and manage transaction boundaries
Persistence	Existing Megara tables and the new JOB_EXECUTION_LOG table	Store operational data and execution history

3.4 How the Solution Works

3.4.1 Reconciliation Reset Job

Purpose

This job clears the reconciliation attributes of daily CSDR penalties that have been amended but are not yet fully reconciled. Specifically, it sets RECONCILIATION_PK to null, resets RECONCILIATION_STATUS to a neutral value, and clears any stored mismatch amounts. The goal is to return these records to a clean baseline so that the next reconciliation engine

run can process them from scratch, without interference from the results of a previous failed attempt.

Eligibility checks

- The penalty's reconciliation status is not already 'Totally Reconciled'. There is no purpose in resetting a record that is already in a clean state.
- The penalty has had at least one amendment applied to it. Resetting a penalty that has not been corrected first would serve no useful function.
- No other job instance is currently processing that record. This concurrency check prevents race conditions when two instances overlap on the same set of penalties.

Execution sequence

Once triggered, whether by an authorized operator through the Megara interface or automatically by the scheduler at its configured time, the job follows this sequence:

1. Query the database to extract all penalties that meet the eligibility criteria above.
2. Open a single database transaction covering all selected records.
3. For each eligible penalty, set RECONCILIATION_PK to null, reset RECONCILIATION_STATUS to 'Not Yet Reconciled', and clear stored mismatch amounts.
4. Update each penalty's workflow state accordingly.
5. Commit the transaction if every operation succeeded. If anything fails, roll back the entire batch automatically.
6. Write a job execution log entry recording the execution mode (MANUAL or SCHEDULED), start timestamp, end timestamp, number of penalties processed, and final status.

3.4.2 Notification Reset Job

Purpose

This job targets a different category of records: penalties where a notification has already been sent to the client but where reconciliation is still showing a mismatch. Clearing the

notification attributes allows these penalties to re-enter the correction workflow and be notified again after successful reconciliation.

Relationship with the Reconciliation Reset Job

These two jobs address different database columns and serve different operational purposes. They can be run in the same daily cycle or on completely separate schedules depending on what the operations team needs on a given day. Neither job calls the other, and neither carries any awareness of the other's execution history or current state.

Execution sequence

7. Select penalties where NOTIFICATION_STATUS is set and RECONCILIATION_STATUS is not 'Totally Reconciled'.
8. Open a single transaction covering all selected records.
9. Clear the notification attributes for each eligible penalty.
10. Update the workflow state for each record.
11. Commit or roll back the entire batch depending on the outcome.
12. Write a job execution log entry with jobType set to NOTIF_RESET.

3.5 UML Models

The UML models in this section give a formal description of the solution from complementary angles. The use case diagram defines the actors and the scope of the system. The state-transition diagram follows the life of a single penalty record through the correction workflow. All models reflect the actual implementation scope, with no speculative features included.

3.5.1 Use Case Diagram

Three actors interact with eight use cases within the Megara post-reconciliation correction module. The interactions between these actors and the system are illustrated in **Figure 24**, which presents the use case diagram. The actors involved are detailed in **Table 7**, while the description of the identified use cases is provided in **Table 8**. The diagram below highlights the actors, the functionalities they access, and the relationships between the different use cases

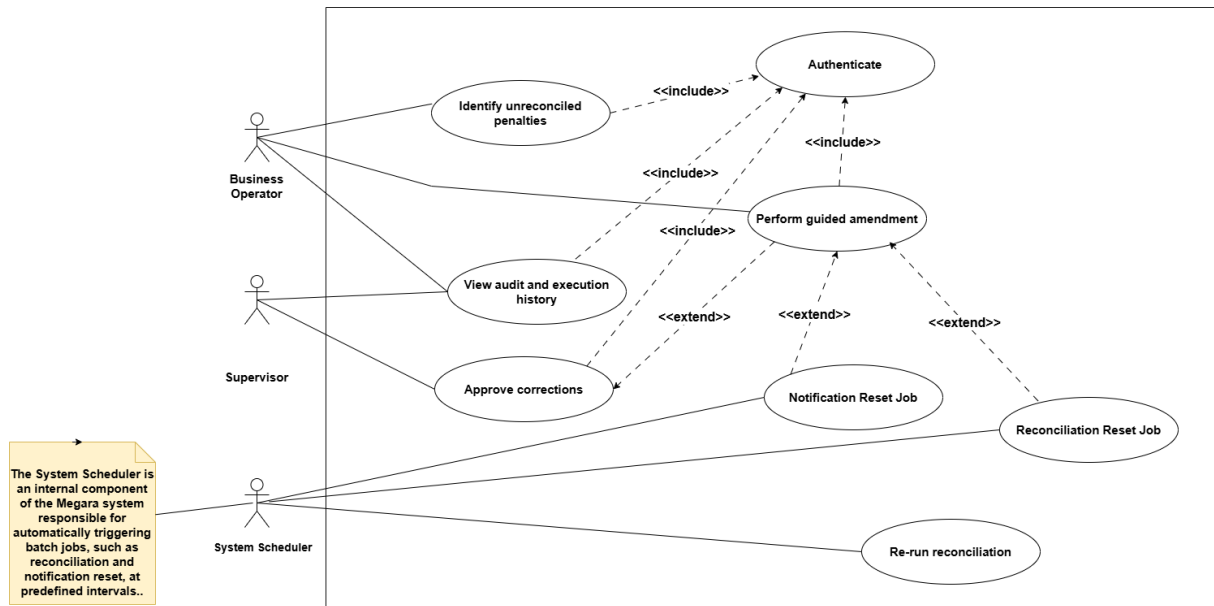


Figure 23: Use Case Diagram of the Post-Reconciliation Correction Module

Table 7: Actors in the Use Case Diagram

Actor	Nature	Role in the system
Business Operator	Human user with standard access	Logs into the system, identifies unreconciled penalties, performs amendments, and consults audit and execution history.
Supervisor	Human user with elevated access	Logs into the system, reviews audit and execution history, and approves sensitive corrections when the four-eyes control principle applies.
System Scheduler	Automated system component	Executes the Reconciliation Reset Job, the Notification Reset Job, and the reconciliation job automatically according to their configured schedules.

Table 8: Description of the Use Cases

ID	Use case	Primary actor	Description
UC0	Log in	Business Operator / Supervisor	Authenticate the user and grant access to the system according to the assigned role and permissions.
UC1	Identify unreconciled penalties	Business Operator	Browse and filter daily penalties that are not fully reconciled, using criteria such as status, date, and counterparty.
UC2	Perform amendment	Business Operator	Correct erroneous data through Market Amendment or Client Amendment, depending on the source and nature of the discrepancy.
UC3	Reconciliation Reset Job	System Scheduler	Reset reconciliation-related attributes for eligible amended penalties so that they can be processed again by the reconciliation engine. This use case may be triggered in the amendment context when required.
UC4	Notification Reset Job	System Scheduler	Reset notification-related attributes for penalties that were already notified and later corrected, so that they can be notified again after successful reconciliation. This use case remains independent from the reconciliation reset process.
UC5	Re-run reconciliation	System Scheduler	Execute the reconciliation job again after the reset operations have completed, allowing corrected penalties to be reprocessed based on the updated data.
UC6	View audit and execution history	Business Operator / Supervisor	Access job execution logs, amendment traces, and operational history for monitoring, compliance, and audit purposes.
UC7	Approve sensitive corrections	Supervisor	Validate sensitive or high-impact amendment corrections before final processing, in accordance with the four-eyes control principle. This use case applies only in specific situations.

3.5.2 State-Transition Diagram

The lifecycle of a single daily penalty within the correction workflow is illustrated through the state-transition diagram presented in **Figure 25**. Each state corresponds to a well-defined

phase in this process, while transitions between states are triggered by specific actions, either human or automated. A detailed description of these states is provided in **Table 9**.

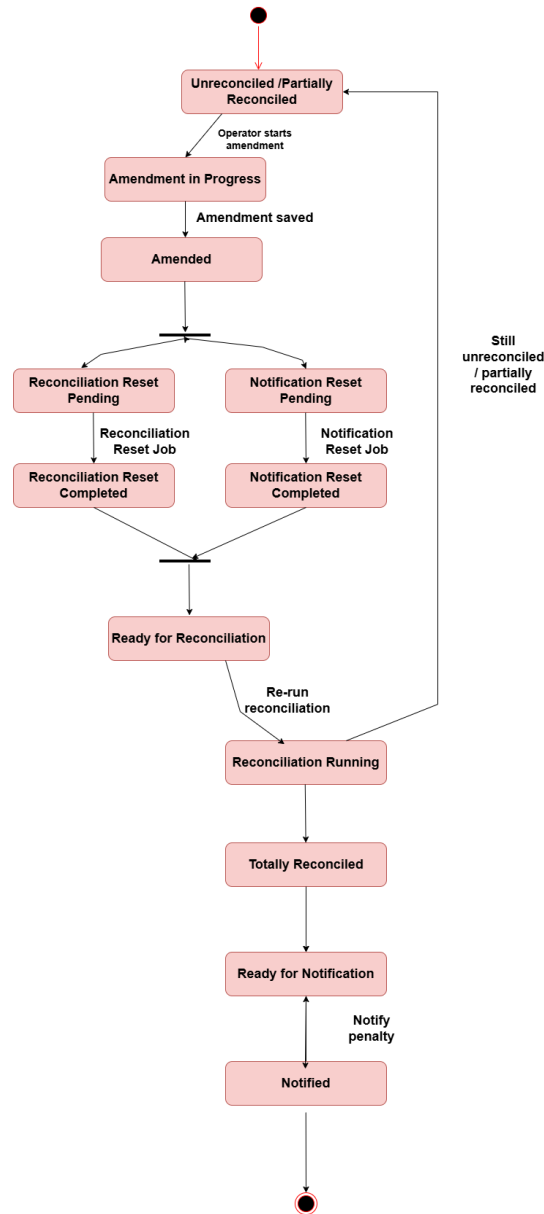


Figure 24: State-Transition Diagram of the Correction Workflow

Table 9: Description of the States in the State-Transition Diagram

State	Description
Unreconciled / Partially reconciled	Initial state after reconciliation results indicate that the penalty still contains discrepancies and requires corrective action.
Amendment in progress	The operator is actively applying a Market Amendment or Client Amendment to correct the penalty data.
Amended	The amendment has been completed and saved successfully, and the penalty is now eligible for reset operations.
Reconciliation reset pending	The penalty is waiting for the Reconciliation Reset Job to clear reconciliation-related fields.
Notification reset pending	The penalty is waiting for the Notification Reset Job to clear notification-related fields.
Reconciliation reset completed	The reconciliation-related fields have been successfully reset.
Notification reset completed	The notification-related fields have been successfully reset.
Ready for reconciliation	Both reset operations have completed, and the penalty is ready to be processed again by the reconciliation engine.
Reconciliation running	The reconciliation job is being executed again on the corrected penalty.
Totally reconciled	The penalty has been successfully reconciled after correction, and no discrepancy remains.
Ready for notification	The penalty is fully corrected and reconciled and is now ready to be notified again.
Notified	The penalty has been notified after successful correction and reconciliation, and the correction process is complete.

3.5.3 Sequence Diagrams of the Reset Process

To better illustrate the dynamic behavior of the system, two sequence diagrams are proposed. These diagrams focus specifically on the execution of the Reconciliation Reset Job, distinguishing between its manual and automatic triggering modes.

Unlike the state-transition diagram, which describes the lifecycle of penalties, the sequence diagrams highlight the interactions between system components during the execution of the reset process.

3.5.3.1 Manual Execution of the Reconciliation Reset Job

The first sequence diagram presented in **Figure 25** describes the manual execution of the Reconciliation Reset Job, initiated by the Business Operator through the Megara system.

The process begins when the operator triggers the reset job. The request is then forwarded to the Reconciliation Reset Service, which is responsible for executing the business logic. The service retrieves the list of eligible penalties from the Database.

If eligible penalties are found, the system iterates over each penalty and resets the reconciliation-related attributes according to predefined rules. Once all updates are performed, the changes are committed to ensure data consistency. The system then returns a success status, indicating that the job has been successfully completed.

In case of failure during execution, the process is aborted, and an error status is returned to the user.

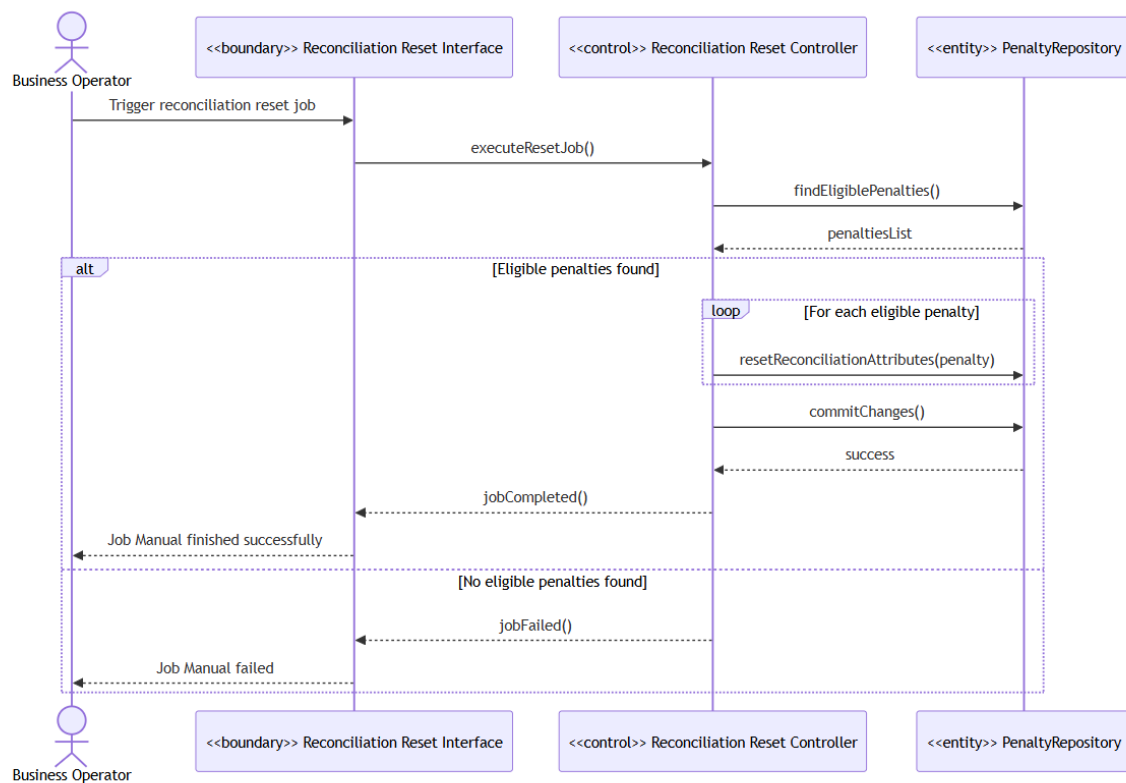


Figure 25: Sequence Diagram of the Manual Execution of the Reconciliation Reset Job

3.5.3.2 Automatic Execution of the Reconciliation Reset Job

The sequence diagram illustrated in **Figure 26** presents the automatic execution of the **Reconciliation Reset Job**, triggered by the **Scheduler**, an internal batch processing component of the Megara system.

At predefined intervals, the Scheduler initiates the Reconciliation Reset Job by invoking the corresponding service. The Reconciliation Reset Service retrieves eligible penalties from the database.

If penalties are available, the system processes each one iteratively by resetting the reconciliation attributes and committing the changes. Upon successful completion, the service returns an execution report indicating success.

If no eligible penalties are found, or if an error occurs during processing, the job returns a failure or no-data status accordingly.

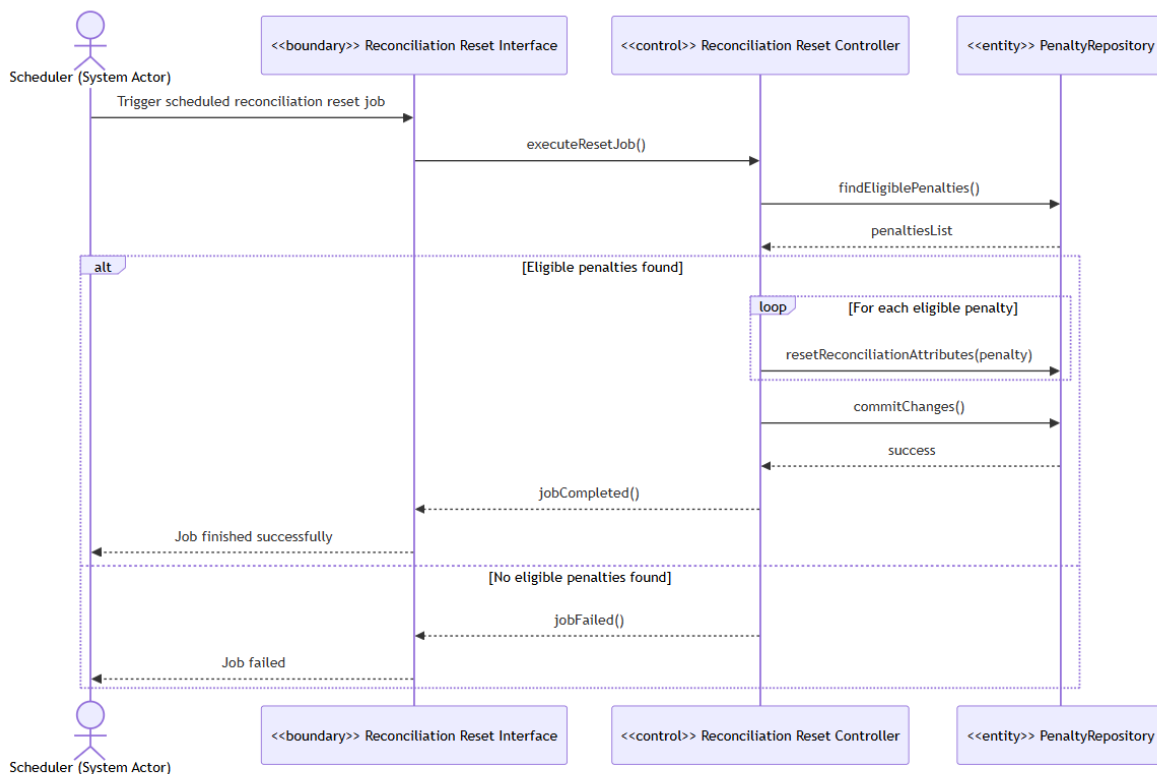


Figure 26: Sequence Diagram of the Automatic Execution of the Reconciliation Reset Job

Conclusion

This chapter translated the operational and regulatory issues identified in Chapter 2 into a practical solution based on two batch jobs: the Reconciliation Reset Job, which clears reconciliation attributes of amended penalties, and the Notification Reset Job, which resets notification attributes for penalties still in mismatch.

The solution was designed to integrate seamlessly into Megara's existing architecture, ensuring transactional safety, proper access control, and full traceability through structured audit logs.

The UML models illustrate these design choices from different perspectives, with the use case diagram defining system interactions and the state-transition diagram describing the lifecycle of a penalty and the role of each job.

The next chapter presents the implementation and testing phases, highlighting how the solution performs in practice in terms of correctness, performance, and audit readiness.

CHAPTER 4: Implementation and Testing

Introduction

Chapter 3 described what was designed and why. This chapter focuses on how the solution was implemented and validated. It begins by presenting the testing methodology followed throughout the project, then explains the eligibility conditions that govern each job, shows the job configurations inside the Megara platform, presents the unit and integration test results, and closes with a complete end-to-end test case that walks through the full correction workflow supported by real screenshots from the platform.

4.1 Testing Approach

4.1.1 Testing Within the Sprint Cycle

VERMEG adopts an Agile approach that blends the structured framework of Scrum with the flexibility of Kanban. In this context, testing is not viewed as a final step in the development cycle; instead, it is integrated throughout the entire process. Test cases are designed alongside development activities and executed as soon as new features are available in the build as summarized in Table 10.

As a result, each acceptance criterion defined in Chapter 3 directly serves as a test condition. This continuous testing approach allows issues to be detected and addressed within the same sprint in which they arise, rather than being postponed to a later validation phase. Consequently, this significantly reduces both the time and effort required for issue resolution.

Table 10: Overview of Testing Activities and Outputs During Agile Sprint Phases

Sprint phase	Testing activity	Output
Sprint planning	Review user stories for testability and agree on acceptance criteria before development starts.	Refined acceptance criteria
Development (days 1 to 4)	Write test cases in parallel with development and identify boundary and negative scenarios.	Test case specifications
Feature complete (days 5 to 8)	Execute test cases on the new build and log any defects with reproduction steps.	Execution results, defect reports
Stabilisation (days 9 to 10)	Perform regression testing on the integrated flows and retest all resolved defects.	Regression report, defect closure
Sprint review	Present test results alongside the feature demo and communicate any residual risks.	Sprint test summary

4.1.2 Testing Levels Applied

The validation of the solution was organized across three complementary testing levels, each addressing a specific type of risk.

Unit testing :

Unit tests were developed by the team to validate the core logic of each job component in isolation. These tests covered key aspects such as eligibility filter evaluation, field-level reset operations, transaction rollback in case of simulated database errors, and method-level access control. Executed with every build, they provided fast and continuous feedback throughout the development process.

Integration testing :

Integration testing focused on ensuring that all components functioned correctly when combined. Particular attention was given to the interactions between the reset jobs and the data access layer, as well as their integration with the audit logging service and the Megara scheduler.

Functional testing

Functional testing was carried out within the Megara test environment to validate the end-to-end behavior of the solution from an operator's perspective. The system was tested using real data through the actual Megara interface. A detailed description of the main end-to-end test scenario is provided in Section 4.5, supported by platform screenshots.

4.2 Job Eligibility Conditions

Before processing any penalty record, each reset job evaluates a minimal set of eligibility conditions. These checks are implemented as simple Java validations at the beginning of the job execution. Their role is only to determine whether a penalty is eligible for the reset operation.

4.2.1 Reconciliation Reset Job Conditions

For a daily penalty to be processed by the Reconciliation Reset Job, the following condition must be satisfied:

```
penalty.getReconciliationStatus() != ReconciliationStatus.TOTALLY_RECONCILED  
|| penalty.getReconciliationPk() != null  
|| penalty.getInternalAmount() != null  
|| penalty.getNotReconciledAmount() != null
```

Once this condition is met, the job resets the reconciliation data by clearing the relevant fields (such as reconciliation status and associated amounts).

The objective is not to select empty records, but to **empty these fields** so the penalty can be processed again through the reconciliation workflow.

4.2.2 Notification Reset Job Conditions

For a daily penalty to be processed by the Notification Reset Job, the following condition must be satisfied:

Condition: A notification has already been initialized for the penalty

```
penalty.getNotificationStatus() != null
```

When this condition is met, the job clears the notification-related fields. This reset enables the system to regenerate and resend the notification if required.

4.3 Job Configuration in Megara

Both jobs are registered as independent entries in Megara's native job scheduler. Their configuration was completed before any testing began, using the standard scheduler administration interface.

4.3.1 Reconciliation Reset Job Configuration

The Reconciliation Reset Job is registered under the reference RECON_RESET_JOB. In the test environment, the job is configured to run automatically each business day at 02:00 as shown in Figure 27.

The screenshot shows the 'Edit : Job Detail' form for a job named 'pen_recon_reset_job'. The form is divided into two main sections: a table at the top and a detailed form below. The table lists the job name, request recovery status (No), and description. The detailed form below contains fields for Name, Group, Job Impl Class, Request Recovery, Description, Job Creation Date, Job Creator User Id, Job Update Date, and Job Updator User Id.

Name	Request Recovery	Description
pen_recon_reset_job	No	Job to reset reconciliation attributes for amended penalties eligible for re-reconciliation.

Name	Group	Job Impl Class
pen_recon_reset_job	DEFAULT	com.vermeg.services.quartzScheduler.impl.JobImpl
Request Recovery	Description	Job Creation Date
No	Job to reset reconciliation attributes for amended penalties eligible for re-reconciliation.	02/03/2026 12:00:12:248
Job Creator User Id	Job Update Date	Job Updator User Id
	02/03/2026 12:00:12:248	

Figure 27:

The trigger configured for this job defines the automatic execution schedule. Figure 28 shows the corresponding trigger settings.:

The screenshot shows the 'View : Monthly Trigger' form for a trigger named 'trg_pen_recon_reset_monthly'. The form is divided into two main sections: a table at the top and a detailed form below. The table lists the trigger name, group, day of month, mis fire instruction, priority, repeat count, start time, status, track execution, and end time. The detailed form below contains fields for Name, Group, Day Of Month, Mis Fire Instruction, Priority, Repeat Count, Start Time, Status, Track Execution, Business Calendar, Description, Trigger Creator User Id, and Trigger Updator User Id.

Name	Group	Day Of Month
trg_pen_recon_reset_monthly	DEFAULT	1

Mis Fire Instruction	Priority	Repeat Count
0	6	-1
Start Time	Status	Track Execution
01/01/2026 02:00:00	Active	Yes
Business Calendar	Description	End Time
Working Days Calendar (optional)	Monthly trigger executing the Reconciliation Reset Job on the first day each month.	
Trigger Creator User Id	Trigger Updator User Id	
admin@ALL	admin@ALL	

Figure 28:

4.3.2 Notification Reset Job Configuration

The Notification Reset Job is registered separately under the reference NOTIF_RESET_JOB. It has its own independent schedule set to 02:30 in Figure 29., thirty minutes after the Reconciliation Reset Job, ensuring that reconciliation resets are always processed first within the daily cycle presented in Figure 30.

Edit Job Detail

Name	Request Recovery	Description
pen_notif_reset_job	No	Job to reset notification status for corrected penalties already notified, enabling re-notification after successful reconciliation.

Page 1 of 1

View Job Detail

Job Detail | Notification Of Failure (0) | Notification Of Success (0) | Trigger (1)

Name	Group	Job Impl Class
pen_notif_reset_job	DEFAULT	com.vermeg.services.quartzScheduler.impl.JobImpl
Request Recovery	Description	Job Creation Date
No	Job to reset notification status for corrected penalties already notified, enabling re-notification after successful reconciliation.	02/03/2026 16:15:30:789
Job Creator User Id	Job Update Date	Job Updater User Id
	02/03/2026 18:15:30:789	

Figure 29:

Page 1 of 1

View Monthly Trigger

Monthly Trigger | Next Executions [50]

Name	Group	Day Of Month
trg_pen_notif_reset_monthly	DEFAULT	1
Mis Fire Instruction	Priority	Repeat Count
0	5	-1
Start Time	Status	Track Execution
01/01/2026 02:30:00	Active	Yes
Business Calendar	Description	End Time
Working Days Calendar (optional)	Monthly trigger executing the Notification Reset Job on the first day of each month.	
Trigger Creator User Id	Trigger Updater User Id	
admin@ALL	admin@ALL	

Figure 30:

4.4 Unit and Integration Testing

4.4.1 Unit Testing

Unit tests verified the core logic of each job in isolation. The key test cases executed for both jobs are detailed in Tables 11 and 12.

Table 11: Unit test cases for the Reconciliation Reset Job

Test ID	Scenario	Expected result	Result
UT-RR-01	Penalty is amended and not reconciled: qualifies for the batch.	Penalty included in the reset batch.	PASS
UT-RR-02	Penalty is already TOTALLY_RECONCILED: must be excluded.	Record skipped, no modification.	PASS
UT-RR-03	Penalty has no amendment on record: must be excluded.	Record excluded from the batch.	PASS
UT-RR-04	RECONCILIATION_PK is set to null after the reset.	Field equals null after execution.	PASS

Table 12: Unit test cases for the Notification Reset Job

Test ID	Scenario	Expected result	Result
UT-NR-01	Penalty is notified and not reconciled: qualifies for the batch.	Included in the reset batch.	PASS
UT-NR-02	Penalty has no notification flag: must be excluded.	Record untouched.	PASS
UT-NR-03	Penalty is fully reconciled despite being notified: excluded.	Record excluded.	PASS

4.4.2 Integration Testing

Integration testing verified that components interacted correctly once assembled, focusing on the boundaries between the reset jobs, the data access layer, the audit logging service, and the scheduler as detailed in Table 13.

Table 13: Integration test cases

Test ID	Components	Scenario	Result
IT-01	ReconResetJob and PenaltyDAO	Reset eligible penalties and verify field values before and after using a direct SQL check.	PASS
IT-02	NotifResetJob and PenaltyDAO	Reset notified penalties and verify that only notification fields were cleared.	PASS
IT-03	ReconResetJob and AuditService	Run the job and verify that JOB_EXECUTION_LOG contains a correctly structured entry.	PASS
IT-04	Scheduler and ReconResetJob	Allow the scheduler to fire the job and verify that executionMode is recorded as SCHEDULED.	PASS
IT-05	Scheduler and NotifResetJob	Verify that the job fires on its independent schedule without affecting the Reconciliation Reset Job.	PASS
IT-06	Both jobs in sequence	Run both jobs in the same cycle and verify two separate log entries with no shared state.	PASS

4.5 End-to-End Test Case: Full Correction Workflow

This section presents the complete functional test case executed on the Megara platform. It covers the full correction workflow for a daily penalty that starts in a partially reconciled state at the end of a month and ends totally reconciled after passing through a FWAM market amendment, the automatic Reconciliation Reset Job, the automatic Notification Reset Job, and the reconciliation engine.

This scenario was chosen because it is the most complete and realistic path through the solution. It exercises every component designed and implemented in this project and reflects the workflow that operations teams will follow in daily production use.

4.5.1 Step 1: Checking Penalties After Monthly Reconciliation

At the end of the month, the reconciliation engine runs and processes all daily penalties. As shown in Figure 31, the penalty list after this run. Two penalties are visible: one that is TOTALLY_RECONCILED and one that is PARTIALLY_RECONCILED.

The screenshot displays the 'Reconciliation' section of the Megara platform. The top navigation bar includes 'Settlement', 'Blocking', 'Notification', 'Position', 'Transaction fees', 'MegaCustody Setup', 'Reports', 'Tools', 'Archive & Purge', and 'System Configuration'. The 'Business Date' is set to 20/10/2025. The main content area shows the 'Reconciliation' process with search criteria 'Incoming Statement Of Penalties' and search results 'Incoming Statement Of Penalties'. The view is 'Incoming Statement Of Penalties'. The 'Reconciliation' process is shown with the following details:

- Process Id: SPIN10001
- Sender Address: CEDELULLDXXX
- Subject To Reconciliation: true
- Report Frequency: MNTH
- Statement Date: 20/10/2025
- Period From: 01/09/2025
- Period To: 30/09/2025

The reconciliation results are displayed in a table:

Reconciled Items	Partially Reconciled Items	Not Reconciled External Items	Not Reconciled Internal Items
Reconciled Items [1]	Partially Reconciled Items [1]	Not Reconciled External Items [0]	Not Reconciled Internal Items [0]

The 'View : Incoming Penalty Details' table shows the following data:

Account DSS	Account Identifier	Account Number	Account Type Code	Amount	C ASD Data Source Scheme	C A
✓ null	99099	99099	null	42.06	CES	CES
✓ null	99099	99099	null	12.06	CES	CES

Figure 31:

Opening the detail of the first penalty (Pena 1) confirms why it is totally reconciled As shown in Figure 32: the amount received in the monthly penalties report matches the internal client amount exactly (42.06). There is no discrepancy.

Page 1 of 1

Disclaimer: All data is subject to verification. 1 of

View : Incoming Penalty Details

Account DSS	Account Identifier	Account Number
null	99099	99099
Account Type Code	Amount	C ASD Data Source Scheme
null	42.06	
C ASD Identifier Code	C ASD Name And Address	C ASD Proprietary Code
CEDELULLXXX		
Calculation Method	Cash Account Inter Number	Cash Account Number
SECU		99099
Computed Amount Currency	Computed Amount Flag	Global Net Amount
EUR	true	54.12
Global Net Amount Curr Code	Internal Amount	Is Parent Penalty
EUR	42.06	false
Not Reconciled Amount	Number of days	Payment Date
0	1	
Payment Time	Penalty Common Reference	Penalty Pk
	CE529500000991	
Penalty Reference	Penalty Status Code	Penalty Status DSS
NC6529500000991		
Penalty Type	Period From	Period To
SEFP	01/09/2025	30/09/2025
Reconciliation Failure Reason	Reconciliation Status	Repa Data Source Scheme
	totally reconciled	
Repa Identifier Code	Repa Name and Address	Repa Proprietary Code
Sender Address	Status	FOOTPRINT
CEDELULLXXX	Valid	

Figure 32:

Opening the detail of the second penalty (Pena 2) shows the source of the partial reconciliation As shown in Figure 33 . The amount received in the monthly penalties reconciliation SWIFT message is 32.06, but the internal client amount is 12.06. This difference of 20.00 is the discrepancy that caused the PARTIALLY_RECONCILED status and that must now be corrected.

Page 1 of 1

Displaying 1 - 1 of

View : Incoming Penalty Details

Account DSS	Account Identifier	Account Number
null	99099	99099
Account Type Code	Amount	C ASD Data Source Scheme
null	12.06	
C ASD Identifier Code	C ASD Name And Address	C ASD Proprietary Code
CEDELULLXXX		
Calculation Method	Cash Account Inter Number	Cash Account Number
SECU		99099
Computed Amount Currency	Computed Amount Flag	Global Net Amount
	true	54.12
Global Net Amount Curr Code	Internal Amount	Is Parent Penalty
	32.06	false
Not Reconciled Amount	Number of days	Payment Date
20	1	
Payment Time	Penalty Common Reference	Penalty Pk
	CE529500000992	1492934281
Penalty Reference	Penalty Status Code	Penalty Status DSS
NC6529500000992		
Penalty Type	Period From	Period To
SEFP	01/09/2025	30/09/2025
Reconciliation Failure Reason	Reconciliation Status	Repa Data Source Scheme
	Partially Reconciled	
Repa Identifier Code	Repa Name And Address	Repa Proprietary Code
Sender Address	Status	
CEDELULLXXX	Valid	

Figure 33:

4.5.2 Step 2: Receiving and Applying the FWAM Amendment

The CSD sends a FWAM (Forward Amendment) SWIFT message to correct the market-side data for the partially reconciled penalty. A FWAM message is used by the CSD to transmit corrected penalty details after the original daily report has already been processed. In this case, the FWAM carries the corrected bilateral amount of 12.06, which brings the market amount in line with the internal client amount.

Megara processes the FWAM and creates a new penalty detail record with the status AMENDED. The following screenshot shows the amended penalty detail with the corrected bilateral amount of 12.06 as received in the FWAM SWIFT message As shown in Figure 34.

App Reference	Main Reference	Client Reference	Bilateral Net Amount	Bilateral Net Amount Curr	Bic	Bilateral N
CPENA00002233	SCI000019411	999CSDR599	12.08	EUR	CEDELULLXXX	
CPENA00002231	SCI000019411	999CSDR999	42.05	EUR	CEDELULLXXX	
CPENA00002229	SCI000019402	887CSDRT881	12.06	EUR	CEDELULLXXX	
CPENA00002229	SCI000019402	887CSDRT881	32.06	EUR	CEDELULLXXX	
CPENA00002225	SCI000019639	1008CSDR1	32.15	EUR	CEDELULLXXX	
CPENA00002225	SCI000019383	1017CSDRT1	-32.04	EUR	CEDELULLXXX	
CPENA00002221	SCI000019333	1016CSDRT3	-32.07	EUR	CEDELULLXXX	
CPENA00002217	SCI000019371	1016CSDRT4	-32.07	EUR	CEDELULLXXX	
CPENA00002215	SCI000019371	1016CSDRT4	-32.15	EUR	CEDELULLXXX	
CPENA00002213	SCI000019371	1016CSDRT4	-22.17	EUR	CEDELULLXXX	

Address	Si...	Statement Date	Status	Tasset Description	Tasset ISIN	Td Ack Status Date	Td Ack Status Time
DXXX	---	15/09/2025	Amended	CHINA HONGQIAO GROUP LTD HONC	XS2968971678	15/09/2025	08:21:01:000
DXXX	---	15/09/2025	Valid	CHINA HONGQIAO GROUP LTD HONC	XS2968971678	15/09/2025	08:21:01:000

Figure 34

At this stage the data is corrected, but the reconciliation-related fields and notification fields from the previous failed reconciliation run are still populated. These must be cleared by the two reset jobs before the reconciliation engine can process the penalty again cleanly.

4.5.3 Step 3: Checking Fields Before the Reset Jobs Execute

Before the scheduled execution of the reset jobs, the fields that need to be cleared are still populated. The Figure 35 shows the penalty record at this stage, with the reconciliation key, reconciliation status, and notification status all containing values from the previous reconciliation run.

The screenshot shows a web application window titled 'Search For Edit Incom Pena Details : Incoming Penalty Details'. It contains a table of penalty records and a detailed view of a selected record.

Account DSS	Account Identifier	Account Number	Account Type Code	Amount	C ASD Data Source Scheme	C ASD Identifier
☒	null	99099	95099			CEDELULL
☐	null	99099	99099			CEDELULL
☐	null	99099	99099		12.06	CEDELULL
☐	null	99099	99099		42.06	CEDELULL
☐	null	88088	88088		12.06	CEDELULL
☐	null	88088	88088		42.06	CEDELULL
☐	null	12A09	12A09		22.06	CEDELULL
☐	null	12A09	12A09		32.06	CEDELULL
☐	null	12A09	12A09		32.06	CEDELULL
☐	null	12A09	12A09		32.07	CEDELULL

Below the table is a detailed view of a selected record:

Penalty PK	1492934281	Penalty Common Reference	C6529500000992	Penalty Reference	NC629500000992	2023-10-20, v2023...
Account Identifier	99099	Account Number	88089	Amount	12.06	
Global Net Amount Curr Code	EUR	Global Net Amount	54.12	Not Reconciled Amount		
Internal Amount	32.06	Reconciliation Status	Partially Reconciled	Payment Date		

Figure 35: Penalty fields still populated before the reset jobs execute

4.5.4 Step 4: Automatic Execution of the Reset Jobs

At their configured schedule times, the Reconciliation Reset Job fires at 02:00 and the Notification Reset Job fires at 02:30. Both jobs identify the partially reconciled amended penalty as eligible and process it. As shown in Figure 36 the execution log entries confirming that both jobs ran successfully.

The screenshot shows a web application window titled 'Dashboard > Find / Create Job > Track Job Execution'. It contains a table of job execution logs.

Job Name	Trigger Name	Status	Cause Of Failure	Execution Start Date	Execution End Date	Next Execution Start Date
☒ pen_reconciliation_job	trg_pen_reconciliation_monthly	Finished		01/03/2026 06:03:05	01/03/2026 06:06:30	01/04/2026 06:03:00
☐ pen_notif_reset_job	trg_pen_notif_reset_monthly	Finished		01/03/2026 02:00:00	01/03/2026 02:03:00	01/04/2026 06:00:00
☐ pen_recon_reset_job	trg_pen_recon_reset_monthly	Finished		01/03/2026 02:00:00	01/03/2026 02:02:30	01/04/2026 02:00:00

Figure 36:

After the jobs execute, the reconciliation-related fields and notification fields of the amended penalty are cleared. As shown in Figure 37 the penalty record after both reset jobs have completed. The reconciliation key is now null, the reconciliation status has been reset, and the notification status has been cleared.

Dashboard > Penalties Process > Single Penalties > Edit Client Penalty Details

Search For Edit Client Pen Details : Client Penalty Details

Search Result : Client Penalty Details

Penalty Common Reference	Penalty Reference	Account Number	Account Identifier	Client securities account	Computed Amount	Computed
☐ C6529550000992	NC6529500000992	90009	99009	TEST85	12.06 EUR	
☐ C6529550000992	NC6529500000992	90009	99009	TEST85	32.06 EUR	
☒ C6529550000991	NC6529500000991	90009	99009	TEST85	42.06 EUR	
☐ C6529550000881	NC6529500000991	88088	88088	TEST85	12.06 EUR	
☐ C6529550000881	NC6529500000881	88088	88088	TEST85	42.06 EUR	
☐ C6529550000882	NC6529500000882	88088	88088	TEST85	32.08 EUR	
☐ C3509882822056	NC509882822056	12A09	12A09	CAFG02	32.15 EUR	
☐ C6529522210412	NC6529522610412	56F98	55F98	TEST85	-22.04 EUR	
☐ C6522522810412	NC6529522610412	56F98	55F98	TEST85	-12.04 EUR	
☐ C6529522810411	NC6529522610411	45921	45921	TEST10	-32.04 EUR	

Page 1 of 14

Displaying 1 - 10 of 137

Edit Client Penalties Details : Client Penalty Details

Computed Amount Currency EUR	Penalty Reference NC6529500000991	Penalty Common Reference C6529500000991
Penalty Indicator FWIS	Calculation Method SECU	Computed Amount 42.06
Number of days	Penalty Type SEFP	Penalty Date 15/09/2025
Is Notified By Daily Report ☐	Is Notified By Daily Swift ☐	Is Notified By Monthly Report ☐
Is Notified By Monthly Swift ☐	Reconciliation Status	

Figure 37:

Figure 38 shows the updated state of the penalty from the penalties management screen. The PARTIALLY_RECONCILED penalty now appears without reconciliation attributes, confirming it has been successfully reset and is ready for re-reconciliation.

Search For Edit Incom Pena Details : Incoming Penalty Details

Search Result : Incoming Penalty Details

Account DSS	Account Identifier	Account Number	Account Type Code	Amount	C ASD Data Source Scheme	C ASD Id
☒ null	99099	95099	null			CEDELUL
☐ null	99099	95099	null			CEDELUL
☐ null	99099	99099	null	12.06		CEDELUL
☐ null	99099	99099	null	42.06		CEDELUL
☐ null	88088	88098	null	12.06		CEDELUL
☐ null	88088	88088	null	42.06		CEDELUL
☐ null	12A09	12A09	null	22.06		CEDELUL
☐ null	12A09	12A09	null	32.06		CEDELUL
☐ null	12A09	12A09	null	32.06		CEDELUL
☐ null	12A09	12A09	null	32.07		CEDELUL

Page 1 of 11

Displaying 1 - 10 of 103

Edit Incoming Penalty Details : Incoming Penalty Details

2023-10-20, v2023...

Penalty PK	Penalty Common Reference	Penalty Reference
	C6529500000992	NC6295000000992
Account Identifier	Account Number	Amount
99099		12.06
Global Net Amount Curr Code	Global Net Amount	Not Reconciled Amount
EUR		
Internal Amount	Reconciliation Status	Payment Date

Figure 38:

Internally, penalties without a reconciliation status are treated as unreconciled items waiting to be processed. The following screenshot Figure 39 illustrates how these records appear in the internal reconciliation view, confirming they are correctly identified as pending items for the next reconciliation run.

Settlement > Penalties Process > Reconciliation

Search Criteria : Incoming Statement Of Penalties

Search Result : Incoming Statement Of Penalties

View : Incoming Statement Of Penalties

Incoming Statement Of Penalties [Notifications In (1)]

Process Id	Sender Address	Subject To Reconciliation
SPIN10001	CEDELULLDXXX	true
Report Frequency	Statement Date	Period From
MONTH	28/10/2025	01/09/2025
Period To		30/09/2025
Reconciled Items	Partially Reconciled Items	Not Reconciled External Items
Reconciled Items [0]	Partially Reconciled Items [0]	Not Reconciled External Items [0]
		Not Reconciled Internal Items
		Not Reconciled Internal Items [2]

View : Incoming Penalty Details

Account DSS	Account Identifier	Account Number	Account Type Code	Amount	C ASD Data Source Scheme	C A
☐ null	99099	99099	null	42.06		GER
☐ null	99099	99099	null	12.06		CE

Page 1 of 1

Displaying 1 - 2 of 2

Figure 39:

4.5.5 Step 5: Automatic Re-Reconciliation and Final State

On the first business day of the new month, the reconciliation job runs automatically as part of the standard daily processing cycle. Figure 40 shows that both penalties are processed, including the previously partially reconciled one that has now been corrected and reset, and confirms the successful execution of the job.

The screenshot displays a web application interface for managing penalties. At the top, there are search criteria and results for 'Incoming Statement Of Penalties'. Below this, a 'View : Incoming Statement Of Penalties' section shows various filters and counts. The 'Reconciled Items' count is 2, while 'Partially Reconciled Items', 'Not Reconciled External Items', and 'Not Reconciled Internal Items' are all 0. A table titled 'View : Incoming Penalty Details' lists two items with their respective account details and amounts.

Account DSS	Account Identifier	Account Number	Account Type Code	Amount	C ASD Data Source Scheme	C A
☐ null	99099	99099	null	42.06		CEE
☐ null	99099	99099	null	12.06		CEE

Page 1 of 1
Displaying 1 - 2 of 2

Figure 40:

Both penalties are now TOTALLY_RECONCILED. The corrected penalty has been matched successfully: the amended market amount of 12.06 now matches the internal client amount of 12.06, and a new reconciliation key has been assigned. The correction cycle is complete.

4.6 Non-Regression Verification

After completing the functional tests, a non-regression verification confirmed that the addition of the two reset jobs had not disturbed any existing Megara functionality. The following areas were checked:

- The reconciliation engine continued to process penalties correctly after a reset, producing accurate reconciliation results based on the corrected data.

- Monthly penalty aggregation was not affected. The monthly totals continued to reflect the correct sum of daily penalties according to their reconciliation status at the time of aggregation.
- The amendment history screen continued to display the correct before and after values for each amendment with no data loss.
- Existing scheduler jobs continued to run on their configured schedules without any interference from the two new job entries.
- The penalty search and filter screens continued to function correctly across all filter criteria.

No issues were identified during this verification. The existing platform behavior was not impacted in any way.

Conclusion

This chapter covered the implementation and validation of the two reset jobs. The eligibility conditions governing each job were presented both in plain language and as Java conditions, making the decision logic transparent and auditable. The job configurations were shown as they appear in the Megara scheduler, confirming that both jobs are registered independently with separate schedules and access controls.

The unit and integration tests verified the behavior of each component in isolation and in combination. The end-to-end test case then walked through the complete correction workflow on a real partially reconciled penalty, from the reception of a FWAM amendment through the automatic reset jobs and re-reconciliation, with each step documented by a screenshot from the Megara platform. The solution is ready for production deployment.

Conclusion

This project was carried out within the context of post-trade operations in financial markets, with a particular focus on the management of CSDR settlement penalties within the Megara platform developed by VERMEG. The primary objective was to analyze the limitations of the existing post-reconciliation correction process and to design a solution capable of improving its efficiency, reliability, and traceability.

The initial analysis revealed that the current workflow relied heavily on manual intervention. Although functional, this approach presented several challenges, including time-consuming operations, increased risk of human error, limited scalability, and insufficient traceability for audit and compliance purposes. These limitations highlighted the need for a more structured and automated solution.

To address these issues, a solution based on two independent batch jobs was proposed and implemented: the Reconciliation Reset Job and the Notification Reset Job. These components automate the reset of reconciliation and notification attributes for eligible penalties, enabling the system to reprocess corrected data in a controlled and consistent manner. The solution was designed to integrate seamlessly into the existing Megara architecture, respecting both technical constraints and regulatory requirements.

The implementation phase confirmed the feasibility and effectiveness of the proposed approach. Through unit testing, integration testing, and end-to-end validation, the solution demonstrated its ability to reduce manual workload, improve processing consistency, and ensure data integrity through transactional mechanisms. The introduction of structured execution logs also significantly enhanced traceability, supporting audit and compliance needs.

Beyond its technical contributions, this project provided valuable insights into the challenges of designing reliable and scalable solutions within a complex financial system. It emphasized the importance of balancing automation with control, ensuring that improvements in efficiency do not compromise data integrity or regulatory compliance.

However, the solution also revealed certain limitations, particularly regarding the continued dependence on manual analysis for discrepancy resolution and the absence of

intelligent mechanisms for automated decision support. These aspects represent potential areas for future enhancement.

In conclusion, the project successfully achieved its objectives by delivering a practical and effective solution that improves the post-reconciliation correction process within Megara. It contributes to enhancing operational efficiency, system reliability, and regulatory compliance, while providing a solid foundation for future developments in automated financial processing systems.

From a future perspective, several improvements can be considered to further enhance the proposed solution. First, the integration of intelligent decision-support mechanisms, such as rule-based engines or machine learning models, could help automate the identification and classification of reconciliation discrepancies, thereby reducing the need for manual analysis. Second, the implementation of a real-time monitoring dashboard would allow users to track penalty processing, job execution status, and anomalies more efficiently. Additionally, extending the solution to support a wider range of post-trade scenarios and regulatory frameworks could increase its scalability and adaptability.

From a technical standpoint, further optimization of batch performance and the introduction of parallel processing strategies could improve execution time for large data volumes. Moreover, enhancing logging and audit capabilities with more advanced reporting tools would strengthen compliance and operational transparency. Finally, integrating alerting mechanisms and notification systems could ensure proactive issue detection and faster resolution.

These perspectives open the way for transforming the current solution into a more autonomous, scalable, and intelligent system, capable of meeting the evolving demands of financial market infrastructures.

Webographie

- [1] VERMEG official website, *About us*, available at: <https://www.vermeg.com> (accessed on April 3, 2026)
- [1] VERMEG official website, *About us / Leadership*, available at: <https://www.vermeg.com> (accessed April 3, 2026)
- [2] TechTarget, *Waterfall model*, available at: <https://www.techtarget.com> (accessed April 3, 2026)
- [3] Medium, *Agile methodology*, available at: <https://medium.com> (accessed April 3, 2026)
- [4] DoneTonic, *Scrum Process*, available at: <https://donetonic.com> (accessed April 3, 2026)
- [5] Atlassian, *Jira Software: Kanban Boards*, available at: <https://www.atlassian.com/software/jira> (accessed April 3, 2026)
- [6] Microsoft, *Microsoft Teams*, available at: <https://www.microsoft.com/microsoft-teams> (accessed April 3, 2026)
- [7] SWIFT, *MyStandards*, available at: <https://www.swift.com/products/mystandards> (accessed April 3, 2026)
- [8] SoapUI, *SoapUI Product Overview*, available at: <https://www.soapui.org> (accessed April 3, 2026)
- [9] WinSCP, *WinSCP Official Website*, available at: <https://winscp.net> (accessed April 3, 2026)
- [10] ESMA, *CSDR Regulatory Technical Standards (RTS)*, available at: <https://www.esma.europa.eu/document/csd-rts> (accessed April 3, 2026)

Abstract

This project was carried out within the framework of a work-study program at VERMEG and focuses on improving the post-reconciliation correction process of CSDR settlement penalties within the Megara platform. In financial institutions, the management of settlement penalties is a critical operation that requires accuracy, traceability, and compliance with regulatory requirements.

The existing process for correcting reconciliation discrepancies relied heavily on manual operations, which led to inefficiencies, increased risk of human error, and limited scalability. To address these limitations, this project proposes a solution based on the automation of key steps in the correction workflow.

The proposed solution introduces two independent batch jobs: the Reconciliation Reset Job and the Notification Reset Job. These components automate the reset of reconciliation and notification attributes for eligible penalties, allowing the system to reprocess corrected data in a controlled and consistent manner. The solution was designed to integrate seamlessly into the existing Megara architecture while ensuring data integrity through transactional mechanisms and improving traceability through structured execution logs.

The implementation and testing phases demonstrated the effectiveness of the solution in reducing manual workload, improving processing reliability, and enhancing auditability. Although the solution does not eliminate the need for manual analysis, it significantly optimizes the correction process and provides a solid foundation for future enhancements.

Keywords

CSDR, Settlement Penalties, Reconciliation, Automation, Batch Processing, Financial Systems, Megara Platform, Data Integrity, Traceability

Résumé

Ce projet a été réalisé dans le cadre d'un stage en alternance chez VERMEG et porte sur l'amélioration du processus de correction post-réconciliation des pénalités CSDR au sein de la plateforme Megara. Dans les institutions financières, la gestion des pénalités de règlement-livraison constitue une opération critique nécessitant précision, traçabilité et conformité aux exigences réglementaires.

Le processus existant de correction des écarts de réconciliation reposait principalement sur des opérations manuelles, entraînant des inefficacités, un risque accru d'erreurs humaines et une faible capacité de passage à l'échelle. Afin de remédier à ces limitations, ce projet propose une solution basée sur l'automatisation des étapes clés du workflow de correction.

La solution proposée introduit deux traitements batch indépendants : le Reconciliation Reset Job et le Notification Reset Job. Ces composants permettent d'automatiser la réinitialisation des attributs de réconciliation et de notification pour les pénalités éligibles, afin de relancer le traitement dans des conditions fiables et cohérentes. La solution a été conçue pour s'intégrer naturellement dans l'architecture existante de Megara, tout en garantissant l'intégrité des données grâce à des mécanismes transactionnels et en améliorant la traçabilité via des journaux d'exécution structurés.

Les phases d'implémentation et de tests ont démontré l'efficacité de la solution en termes de réduction de la charge manuelle, d'amélioration de la fiabilité des traitements et de renforcement de l'auditabilité. Bien que l'analyse des écarts reste partiellement manuelle, la solution permet d'optimiser significativement le processus de correction et constitue une base solide pour de futures évolutions.

Mots-clés

CSDR, pénalités de règlement, réconciliation, automatisation, traitement batch, systèmes financiers, plateforme Megara, intégrité des données, traçabilité

